A Comprehensive Security Framework For Digital Signatures: Full Version

Paul Gerhart[2], Paul Rösler[1], Dominique Schröder[2], and Julian Thomas[1]

[1] Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
{julian.thomas, paul.roesler}@fau.de
[2] TU Wien, Austria
{paul.gerhart, dominique.schroeder}@tuwien.ac.at

Abstract. Digital signatures are one of the most extensively studied cryptographic primitives, and their security has been a focus of research for the past several decades. While the original security definition, considering existential unforgeability under an adaptive chosen message attack established in 1984 (Goldwasser, Micali, and Rivest), remains the de facto standard, alternative notions have been introduced to address different threat models. Recent work (Jackson et al. CCS'19, Cremers et al. S&P'21, Don et al. CRYPTO'24) revisits this topic, considering security goals beyond unforgeability and corresponding security compilers to achieve these notions. On the other hand, the landscape of adversary models and the corresponding security compilers remains sparse, with open gaps between the security notions and solutions, as well as their relations.

This paper addresses these gaps by presenting a systematic approach to an abstract interface for modeling adversaries against signature schemes. Our method generalizes security definitions and their underlying game-based experiments by giving adversaries oracle access to *all possible* input variables of this interface: messages, randomness, and keys. Based on this, we perform a systematic and formal analysis of the corresponding assumptions as well as generic security compilers to regain security for various notions. In addition, we establish a unified framework for the security of signature schemes, combining our results about the adversary models with security goals of unforgeability and beyond. As an artifact, we provide a website with which every concrete signature security definition from our generalized framework can be dynamically extracted and illustrated. Thereby, we harmonize and tame the complexity of the large zoo of configurable definitions based on all 19200 combinations of classical and extended security goals and adversarial models.

Keywords: Digital Signatures · Adversary Models · Security Goals.

1 Introduction

Digital signatures, first introduced by Diffie and Hellman [24], serve as the digital equivalent of handwritten signatures, allowing anyone with a signature σ, a message m, and a public key pk to verify the validity of σ on m under pk.

Generating σ requires knowledge of the corresponding private key sk. The basic security concept for digital signatures, ensuring that an adversary with access to a signing oracle cannot forge new signatures, was introduced by Goldwasser, Micali, and Rivest in 1984 [41] and remains the de facto standard. Digital signatures are used by billions of people every day, with many schemes standardized (e.g., [73]), and are among the most studied and well-understood cryptographic primitives.

Special Properties. These seminal works have inspired extensive follow-up research in several directions, including improving efficiency [51], developing signatures based on alternative computational assumptions [43], understanding the minimal requirements for constructing them [9], and, more recently, finding post-quantum secure schemes [25,32,38]. Further research has also examined the security of signatures in various settings, exploring both *weaker and stronger adversaries* tailored to specific applications:

(Non-adaptive) message attacks: Weaker security notions include schemes that are only secure against non-adaptive message attacks, uniformly randomly chosen messages, or selective security where the attacker must commit to a message before learning any relevant information, among others [41]. These notions are often used in identification schemes [65,72,11] where the attacker cannot control the message, with the primary goal of constructing more efficient schemes.

Strengthened resilience: Stronger security notions cover attacks against the used sources of randomness [40,29,18,78,69,20,23,35,22] or the bounded [3,50,26,36,60,16] or full [10,1,27,28,76,79,64] leakage of internal secrets via side channels. Signature schemes that are secure against these threats can be used in untrusted environments, in publicly accessible computers, or in settings in which inputs from external sources or the computation unit itself are unreliable.

Another recent line of work extended the landscape of *security goals*:

Beyond Unforgeability: The classic security goal for signatures is unforgeability, which essentially states that an adversary cannot create new valid message signature pairs [42]. However, a growing number of papers [14,6,74,61,68,15,47,17,21] present practical examples where unforgeability alone is insufficient. This leads to security goals beyond unforgeability: *Exclusive Ownership*, which prevents signatures from being valid under different public keys; *Message Bounding*, which prohibits signatures from being valid for two different messages; and *Non-Resignability*, which stops adversaries from signing messages without knowing them. We provide the details in Section A.2 and Section 4.2. To achieve these security goals, Cremers et al. [21] propose a generic security compiler under adaptive message attacks.

Scope. Despite extensive literature offering numerous models and solutions, we are currently lacking a consistent and unified security framework. The diversity of approaches and solutions makes it challenging to combine them trivially. Our goal is to develop a framework that encompasses a comprehensive and consistent

adversary model capable of capturing multiple attack vectors simultaneously, along with corresponding generic solutions to achieve the resulting notions of signature security. While we do revisit existing generic solutions in a systematic and generalized way, our goal is not to propose new countermeasures for these attacks, but to unify, formalize, and analyze the existing landscape within our model. For theory, such a framework is valuable as it provides insights into relationships and new definitions, facilitates easier combination and comparison of security notions, and strengthens our overall understanding. From a practical standpoint, we clarify security assumptions and outline the conditions under which digital signatures can remain secure against a broad range of real-world threats. This enables us to characterize both the limitations of current security models and the fundamental cryptographic mechanisms required to mitigate bad randomness and key exposure threats. By establishing a rigorous framework that explicitly separates trusted and untrusted components, we contribute to a more nuanced and practically relevant understanding of digital signature security.

Approach. The first step towards presenting a comprehensive security framework is a generalized computational model of the underlying cryptographic algorithms (see Figure 2 and Section 3.1), which we call *security interface*. We systematically identify and generalize the parameters of signature schemes and categorize them regarding their cryptographic properties, enabling us to derive potential attack vectors, including input/output attacks, attacks against randomness, and key exposure. From that, we create a generalized security definition (see Section 3.2), under which we revisit existing compiler approaches for bad randomness and key exposure in a systematic and generalized way, and show how they fit naturally into our generic interface (see Section 3.3). This detailed approach gives an in-depth perspective on signature schemes, related algorithms, enabling the refinement of trust assumptions as well as requirements for randomness within different components. Subsequently, we instantiate the security interface for signature schemes to obtain a comprehensive and concrete *security framework* (see Section 4). This extends the classical framework from Goldwasser, Micali, and Rivest [41], which considers message attacks, by randomness and key leakage attacks. Further, it includes the security goals beyond unforgeability, namely exclusive ownership, message bounding, and non-resignability, alongside the various notions of unforgeability for signature schemes [47,17,21].

Interactive Web Application Our resulting framework yields a large and expressive design space: it covers 19200 different combinations of adversary models and security goals in a modular and compact formalization. This breadth is one of the strengths of our theoretical contribution, but it also makes the resulting formalization dense and difficult to parse in its raw form. To address this, we complement the theory with an interactive web application, the *Signature Security Generator*, which directly follows from our formalization and provides a compact, dynamic, and easy-to-use interface. The tool allows users to select arbitrary combinations of attack vectors and security goals and immediately obtain the corresponding security definition, experiment, and oracle formalization in a readable form, together

with exportable LaTeX code that can be copied directly into other documents and may facilitate future research building on our framework. In this sense, it is both a practical proof of concept for the usability of our framework and a direct demonstration that a broad and technically rich theory can still be made accessible, inspectable, and extendable in practice. During the review period, we make the application available at `https://signature-security-generator.org/`; access is provided with the username `reviewer` and the password `access`. In addition, we include the source code as an artifact with the submission.

Contributions. We summarize our contributions as follows:

Comprehensive Framework: We present a generalized security interface based on the cryptographic properties of the parameters, unifying message attacks, randomness attacks, and key exposure in Section 3. As an instantiation, we formalize an extended signature security framework for signature schemes, comprehensively combining our adversary models with various notions of security goals for unforgeability and beyond in Section 4.2.

Signature Security Generator: Due to its high degree of generalization, the formalization of our framework is burdened by density and complexity. We therefore provide an interactive web application, which dynamically presents the security definitions, experiments, and oracles compactly. This makes the framework substantially easier to explore, compare, and understand, and turns our formalization into a practical tool rather than a static description.

Security Compilers and Application: We revisit existing compiler approaches for bad randomness and key exposure in a systematic and generalized way, and show how they naturally fit into our generic interface. This yields a unified perspective on prior assumptions, techniques, and their scope, and turns previously separate ideas into a model for obtaining generic solutions. To demonstrate the practical applicability, we instantiate our framework for the standardized Schnorr signature scheme under bad randomness and key exposure, and incorporate the corresponding security compilers.

1.1 Related Work

Diffie and Hellman introduce the idea of digital signatures [24]. Goldwasser et al. [42] present the classical signature security framework, including the common adversary models of adaptive chosen message security and the common security goal of (strong) existential unforgeability. Numerous papers describe security goals beyond unforgeability [14,6,74,61,68,15], with recent works [47,17,21]. We present these security goals in Section A.2, and briefly discuss similarities and differences at the end of the section.

Regarding attack models beyond adaptive chosen message attacks, bad randomness considers flaws in the generation of randomness. Dodis et al. [29] analyze its cryptographic importance, while Daniel Hutchinson [46] considers the generation and evaluation extensively. However, flawed generators [23], even post-standardization [20], necessitate concrete security definitions and solutions. Multiple studies independently model and address adversarial interactions with

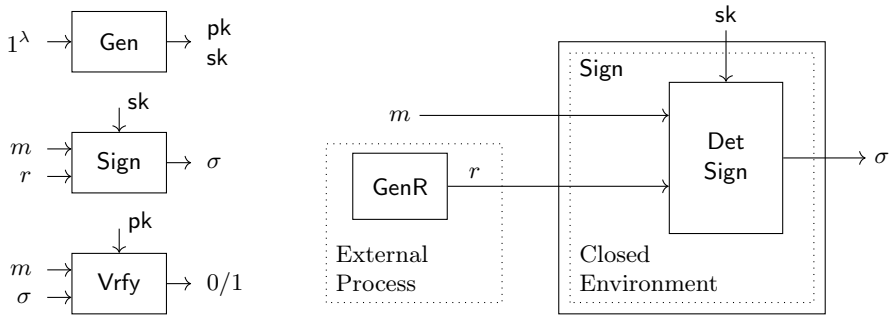
randomness. LaMacchia et al. [58] consider learning the randomness for authenticated key exchange, and Yang et al. [77] expand this for adversarial control of the randomness. Dodis et al. [29] analyze randomness with low entropy for several cryptographic schemes, including signatures. Ristenpart and Yilek [69] and Alekseev et al. [2] consider controlled randomness and present specific notations and solutions. Hanzlik et al. [45] consider a similar adversary model and present a general solution for signatures. Bellare and Tackmann [13] also consider controlled randomness and present a solution utilizing derandomized nonces. Especially, Krzywieck et al. worked on the problem of bad randomness, formalized security models for various systems, and developed appropriate solutions [52,53,54,55,56,57]. Although his formalizations and solutions are specific to the corresponding systems, and thus complicated to generalize, his models serve as inspiration for ours.

Key exposure with (master secret) key insulation is a form of key leakage [34,3,63,50,26,66,44]. Commonly, these models assume the complete leakage of an (ephemeral) key during computation. To maintain security across different computational epochs, they rely on a secure enclave to protect the master key and to derive fresh ephemeral keys. Dodis et al. [27] address this for public-key cryptography, while Dodis et al. [28], Ohtake et al. [64], Zhou et al. [79], and Weng et al. [76] consider the model regarding signatures. Fault attacks include various side-channel attacks that compromise the memory or the computation of a cryptographic scheme [48]. The literature is growing, with both approaches from a more general cryptographic perspective [7,75] and papers on specific primitives, including signature schemes [39,70,67,37]. Recently, we discovered SecurityNotionsZoo [33], a complementary artifact that currently organizes the fully homomorphic encryption literature, but in contrast to our goal, does not derive a unified framework for systematically combining security notions arbitrarily.

To the best of our knowledge, no general and coherent model for adversary models extends beyond the classic framework of Goldwasser et al. [42]. Consequently, various security definitions are often overlooked, and even when addressed in the literature, they are difficult to compare and combine. Consequently, related relationships and solutions may remain unexplored or be difficult to transfer.

2 Preliminaries

Notation. With $x \leftarrow y$ we denote the deterministic assignment of variable y to variable x. We emphasize (uniformly) random sampling from a set S and assignment to a variable x with $x \leftarrow\$ S$. We call a function $f\colon \mathbb{N} \mapsto \mathbb{R}^+$ *negligible* in n if for every $k \in \mathbb{N}$ there exists $n_0 \in \mathbb{N}$ such that $|f(n)| \leq n^{-k}$ holds for every $n \geq n_0$, and we use negl to denote an arbitrary negligible function. λ denotes the security parameter, and when referring to something as negligible, we mean negligible in λ. We use the code-based game-playing framework of Bellare and Rogaway [12] and consider adversaries $\mathcal{A}$ as arbitrary *probabilistic polynomial time* (PPT) algorithms, if not stated otherwise. $\mathcal{A}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}(x_1,\ldots,x_m) = b$ denotes the execution of $\mathcal{A}$ with *Oracle* access to $\mathcal{O}_1,\ldots,\mathcal{O}_n$ using inputs $x_1,\ldots,x_m$,



(a) Visual description of a signature scheme interface.

(b) Closer look into the signing algorithm with a focus on randomness.

and output b. An Oracle is a black box that generates an output for an input according to a formal description. In this work, we use Oracles to model and formalize an adversary's power.

A function $g\colon \{0,1\}^n \to \{0,1\}^{l(n)}$ with uniform random input of length n and a polynomial function l such that $l(n) > n$ is a (deterministic) *pseudorandom generator* PRG if any arbitrary PPT adversary $\mathcal{A}$ can distinguish the output bitstring of g from a real random bitstring only with negligible probability [49]. We will later summarize this concept in a *randomness generation* algorithm GenR, a probabilistic function $g'\colon \emptyset \to \mathcal{D}_R$, which outputs a randomness r uniformly random from the randomness space $\mathcal{D}_R$. A function $f\colon \{0,1\}^\lambda \times \{0,1\}^m \to \{0,1\}^l$ is a (family of) *pseudorandom function(s)* PRF if any arbitrary PPT adversary $\mathcal{A}$ can distinguish f from a real random function only with negligible probability [49].

Digital Signatures. A *digital signature scheme* is a triple of algorithms, guaranteeing a message's integrity, authenticity, and non-repudiation.

Definition 1 (Digital Signatures [49]). *A* (digital) signature scheme $\Sigma = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ *consists of three (probabilistic) polynomial-time* PPT *algorithms:*

- $\mathsf{KGen}\colon \mathcal{D}_\lambda \to \mathcal{D}_\lambda^{\mathsf{sk}} \times \mathcal{D}_\lambda^{\mathsf{pk}}$*: The* key-generation algorithm *outputs a secret key* $\mathsf{sk} \in \mathcal{D}_\lambda^{\mathsf{sk}}$ *and a public key* $\mathsf{pk} \in \mathcal{D}_\lambda^{\mathsf{pk}}$ *depending on the security parameter* λ*.*
- $\mathsf{Sign}\colon \mathcal{D}_\lambda^{\mathsf{sk}} \times \mathcal{D}_M \times \mathcal{D}_R \to \mathcal{D}_\sigma$*: The* signing algorithm *takes as input a secret key* sk*, a message* m *from the message space* $\mathcal{D}_M$*, a randomness* r *from the randomness space* $\mathcal{D}_R$ *and outputs a signature* σ *from the signature space* $\mathcal{D}_\sigma$*.*
- $\mathsf{Vrfy}\colon \mathcal{D}_\lambda^{\mathsf{pk}} \times \mathcal{D}_M \times \mathcal{D}_\sigma \to \{0,1\}$*: The deterministic* verification algorithm *takes as input a public key* pk*, a message* m*, a signature* σ *and outputs a bit* b*, with* $b = 1$ *meaning pair* (m, σ) *is* valid *under* pk *and* $b = 0$ *meaning* invalid*.*

For correctness *we require that there exists a negligible function* negl *such that* $\Pr[(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KGen}(1^\lambda), \mathsf{Vrfy}(\mathsf{pk}, m, \mathsf{Sign}(\mathsf{sk}, m; r)) = 1] \geq 1 - \mathsf{negl}(\lambda)$*, for all* $\lambda \in \mathcal{D}_\lambda$*,* $m \in \mathcal{D}_M$*, and* $r \in \mathcal{D}_R$*.*

Regarding security, we take a deeper look at the practical realization of signatures. The abstract description of signatures from Definition 1 (see Figure 1a) gives the impression that randomness is an external source. At the same time, the security of signature schemes often relies on the assumption that randomness is chosen uniformly at random. However, in practical implementation, the random values are derived by an external process running a randomness generator. This perspective aligns with recent literature [45,23,46]. Figure 1b shows the correspdoning model. Throughout this work, we will extend this point of view concerning security and, building on it, combine several security approaches into a joint theory. We provide a clear and accessible description of the standard adversary models and security definition, as well as the extended security goals from Cremers et al. [21] in Section A.

3 Security Interface

3.1 Model Overview

We aim to systematically and comprehensively extend the classical model of Figure 1b, which results in the generalized model in Figure 2, where we consider three potential attack vectors. The first vector is the classic concept of an adversary learning or controlling external input and output values [42]. Similarly, we model randomness as an untrusted external source, consistent with recent literature [29,69,45,2]. Additionally, we assume an untrusted execution environment for the computation, which may expose internal secrets [27,28]. In summary, we have:

- *Classical Attacks*: Controlling or learning public input and output.
- *Randomness Attacks*: Controlling or learning (external) randomness.
- *Key Exposure*: Full key leakage in an untrusted environment.

We view randomness as an untrusted external source GenR. By separating the randomness generation from any algorithm, we isolate the deterministic subprocedure $\mathsf{DetComp}$, which we regard as any procedure that we deliberately leave unmodified. We allow $\mathsf{DetComp}$ to execute in an untrusted environment—one that may leak all its inputs—to minimize our overall trust assumptions. To compensate, we introduce two trustfully executed helper algorithms, GenTr and GenTsk, alongside a master secret key sk that remains securely insulated. We do so to ensure that our compilers apply to constructions that cannot be modified in retrospect (e.g., established and standardized signature schemes). This approach covers the main algorithm of such constructions. Section 3.3 summarizes Section C and Section D, in which we analyse in detail how to instantiate GenTr and GenTsk to regain security under various threat models for $\mathsf{DetComp}$.

Detailed Model Description. As a basis for our analysis, we aim to provide a generalized and systematic interface for formalizing attack vectors and security definitions, and for deriving generic security compilers. This section considers an

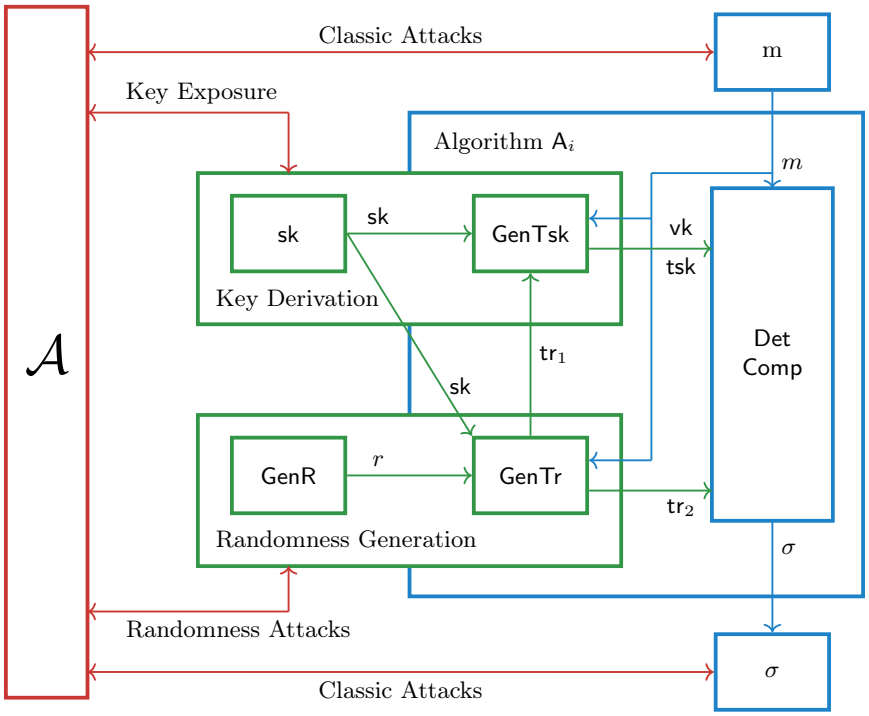


Fig. 2: This generalized and comprehensive security model for signature security extends the classical framework of learning and controlling the message with randomness attacks and key exposure. We assume randomness as an external source GenR generates. GenTr and GenTsk are modification procedures to reobtain security. DetComp resembles the actual (possibly untrusted) computation.

arbitrary cryptographic scheme $\Pi = (\mathsf{A}_1, \ldots, \mathsf{A}_n)$ consisting of multiple (PPT) algorithms, of which an adversary may access a subset. We describe each algorithm by a type annotation of the form $S_\sigma \leftarrow \mathsf{A}_i(S_{\mathsf{sk}}, S_m; S_r)$, meaning that we model it through abstract input and output variables grouped according to common cryptographic properties. In particular, S_m, S_{sk}, S_r, S_{pk}, and S_σ denote the sets of public inputs, secret inputs, randomness, public parameters, and public outputs, respectively. For simplicity and consistency with the signature notation used throughout, we denote the corresponding elements as m, sk, r, pk, and σ.

We split the *Randomness Generation* into the (external) probabilistic generator GenR and the deterministic compiler GenTr. Within GenR, we summarize all algorithms necessary for the initial randomness. To minimize assumptions, we define $\mathsf{GenR}\colon \emptyset \to \mathcal{D}_R$ with $r \in \mathcal{D}_R$. Further, we assume zero trust for GenR [20]. We introduce GenTr, an intermediate algorithm between DetComp and GenR, to recompute the randomness before passing it to DetComp. We assume GenTr is executed trustfully. Multiple existing solutions follow a similar modelling approach [2,13,37,69]. Our goal is to use tr output by GenTr under

the assumption of having "secure" randomness. We achieve this (see Section 3.3) by instantiating GenTr with a PRF. As a result, we define a single compiler as $\mathsf{GenTr}\colon \mathcal{D}_\lambda^{\mathsf{sk}} \times \mathcal{D}_M \times \mathcal{D}_R \to \mathcal{D}_R$, with secret key sk, input (message) m, randomness r, or a subset thereof as input. Similarly, we split the *Key Derivation* into two parts. Since we assume key leakage within $\mathsf{DetComp}$, we introduce GenTsk to derive a temporary secret key tsk (with corresponding public temporary verification key vk) from an incapsulated secret key sk. Again, we assume GenTsk is executed trustfully. Further, if GenTsk is not the identity function, we assume no corruption of sk. As a result, we define $\mathsf{GenTsk}\colon \mathcal{D}_\lambda^{\mathsf{sk}} \times \mathcal{D}_M \times \mathcal{D}_R \to \mathcal{D}_\lambda^{\mathsf{tsk}} \times \mathcal{D}_\lambda^{\mathsf{vk}}$, with $\mathsf{tsk} \in \mathcal{D}_\lambda^{\mathsf{tsk}}$, $\mathsf{vk} \in \mathcal{D}_\lambda^{\mathsf{vk}}$ as well as secret key sk, input m, and (internal) randomness tr. In Section D, we analyze how instantiating GenTsk to achieve security against full leakage of tsk.

3.2 Generalized Security Definition

The *default attack vector*, known as *key-only attack* (KOA) [42], assumes an adversary knows the scheme Π and all public parameters S_{pk}. Our formalization of the *classic attack vector* follows Goldwasser et al. [42] (cf. Definition 5). We call learning the input a known message attack (KMA). Controlling the message is divided into generically choosing (gCMA) and adaptively choosing (aCMA), and for public key schemes, into generic choice (gCMA) and direct choice (dCMA). We note that the same acronym represents the variable choice and the corresponding attack (e.g., we use aCMA for both adaptive message choice and attacks). Regarding the *randomness attack vector*, we follow approaches in modern literature [20,13,2]. We model randomness as an external source that can be fully learned or controlled. Consequently, we adapt the idea from classical security and call learning the randomness known randomness attack (KRA). We divide controlling the randomness into generically choosing (gCRA) and adaptively choosing (aCRA). For public key schemes, we divide them further into generic choice (gCRA) and direct choice (dCRA). In practice, there may be constraints by the functionality of the random generator [20,23,46], and considering arbitrary random values may not adequately represent reality. To capture these cases without complicating the framework, we assume, if needed, that the randomness from either the adversary or the generator is a nonce (a number used only once). *Key exposure* is a form of key leakage that assumes full leakage of the temporary key tsk during the computation of $\mathsf{DetComp}$. Hence, we only model a leakage function $L_{\mathsf{tsk}}\colon \{0,1\}^* \to \mathcal{D}_\lambda^{\mathsf{sk}}$ for tsk. Typically, key exposure assumes stateful counters tied to time periods, resulting in a specification of L_{tsk} that outputs tsk for a given time period. To avoid statefulness, we specify $L_{\mathsf{tsk}}^{m,\sigma}\colon \mathcal{D}_M \times \mathcal{D}_\sigma \to \mathcal{D}_\lambda^{\mathsf{sk}}$, which outputs the corresponding tsk for input m and σ. We present the corresponding oracle formalizations $\mathcal{O}_{\mathsf{Sign}}^{L_{\mathsf{tsk}}^{m,\sigma}}(m,\sigma)$ in Figure 11. Our framework supports fault and fine-grained leakage functions through the same interface that we instantiate here with key exposure. We choose key exposure as the canonical representative of this class, because it captures the modeling principle at the level of abstraction we target. Broader leakage

and fault models require substantially more attack-specific specification of the underlying mechanism, which lies outside the scope of a generalized analysis.

Security Definition. We denote a generic adversary model by $\mathsf{am} = \{\mathsf{mc}, \mathsf{rc}, \mathbb{L}\}$, with arbitrary message choice mc, randomness choice rc, and set of leakage functions $\mathbb{L} = \{L^{m,\sigma}_{\mathsf{tsk}}\}$. Security goals are specific to individual algorithms and schemes; a generalization is not always possible or beneficial. Instead, we use the abstracted security goal sg variable. Then, we write $(\mathsf{sg}\text{-}\mathsf{am})^{\mathcal{A}}_{\Pi}$ to denote that an adversary $\mathcal{A}$ interacts with the security experiment for scheme Π, where $\mathcal{A}$ is granted am, and the scheme is evaluated with respect to sg.

Definition 2 (Generalized Security Definition). *Let $\Pi = (\mathsf{A}_1, \ldots, \mathsf{A}_n)$ be a cryptographic scheme consisting of n algorithms. Let $\{\mathsf{A}_i, \ldots, \mathsf{A}_j\} \subseteq \Pi$ be a subset of algorithms under consideration, and suppose all algorithms in $\Pi \setminus \{\mathsf{A}_i, \ldots, \mathsf{A}_j\}$ are assumed to be secure. We say that Π achieves the* security goal sg *against the subset $\{\mathsf{A}_i, \ldots, \mathsf{A}_j\}$ under the corresponding* adversary models $(\mathsf{am}_i, \ldots, \mathsf{am}_j)$*, or is* sg-$(\mathsf{am}_i, \ldots, \mathsf{am}_j)$*-secure, if for all* PPT *adversaries $\mathcal{A}$, there exists a negligible function* negl *such that*

$$\Pr[(\mathsf{sg} - (\mathsf{am}_i, \ldots, \mathsf{am}_j))^{\mathcal{A}}_{\Pi}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where the probability is taken over the randomness used by $\mathcal{A}$ and the experiment.

The exact formalization of a security experiment depends on the specific security goal. When instantiating an experiment for a scheme $\Pi = (\mathsf{A}_1, \ldots, \mathsf{A}_n)$ with cryptographic algorithms $\mathsf{A}_1, \ldots, \mathsf{A}_n$, we define the interaction between the adversary and the scheme's algorithms using oracles that reflect am. To retain generality in our framework, we specify the input-output behavior of these oracles abstractly and defer their full instantiation until the adversarial model is fixed. An illustrative instantiation is provided for signature schemes in Section 4.2. Each oracle is denoted by $\mathcal{O}^{\mathsf{am}_i}_{\mathsf{A}_i}(\mathsf{sk}, m; r)$, where am_i specifies the adversarial model, and A_i refers to the algorithm for which the oracle simulates interaction. The oracle receives as input a secret key sk, a message m, and randomness r. The adversarial model am_i determines which of these inputs are controlled by the adversary and which are fixed by the experiment. On input $(\mathsf{sk}, m; r)$, the oracle computes $\mathsf{A}_i(\mathsf{sk}, m; r)$ and returns the result. Additionally, it stores the internal state associated with the invocation—including values such as $(r, \mathsf{tr}, \mathsf{sk}, \mathsf{tsk}, m)$—in queues denoted by $\mathcal{Q}^{\{r,\mathsf{tr},\mathsf{sk},\mathsf{tsk},m\}}_{\mathsf{A}_i}$. When context permits, we use abbreviated notation, e.g., $(m_k, r_k, \mathsf{tr}_k) \leftarrow \mathcal{Q}[k]$ to denote the k-th elements of the queues $\mathcal{Q}^{m}_{\mathsf{A}_i}$, $\mathcal{Q}^{r}_{\mathsf{A}_i}$, and $\mathcal{Q}^{\mathsf{tr}}_{\mathsf{A}_i}$, respectively. For transparency, we note that we were not able to identify any non-trivial relations between security definitions (see Section B).

3.3 Generic Security Compilers

In our model, we utilize our security compilers GenTr to secure against bad randomness attacks and GenTsk to secure against key leakage. We now present generic instantiations for both compilers.

Securing Against Bad Randomness. There are numerous existing solutions to secure signature schemes against bad randomness [69,13,5,37,2]. The common approach is to utilize a pseudorandom function PRF with the correct variables as input. We revisit these solutions and systematically analyse them within our generalized interface in Section C. We present the result in Theorem 1. We distinguish between weak non-repeatability, where the output of the PRF is the same for multiple queries with the same input and randomness, and strong non-repeatability, where the output of the PRF is always unique.

Theorem 1 (PRF **Secures Against Bad Randomness).** *Let* $\Pi = (\mathsf{A}_0, \ldots, \mathsf{A}_\lambda)$ *be a cryptographic scheme w.r.t. Model 3.1 satisfying a security goal* sg *against any* PPT *adversary* $\mathcal{A}$ *with adaptive chosen input* aCMA*. Let us further assume that* Π *satisfies this security goal, while* GenTr *computes the identity function (i.e.,* $\mathsf{tr} = r$*),* sk *is secret, and* r *fulfills secrecy, weak (strong) non-repeatability (i.e.,* r *is not trivially leaked by the scheme). Let* $F\colon \{0,1\}^\lambda \times \{0,1\}^* \to \mathcal{D}_R$ *be a* PRF*. Then,* Π *satisfies* sg *against any* PPT *adversary* $\mathcal{A}$ *with* aCMA *and also with adaptive chosen randomness* aCRA*, if we modify* GenTr *into*

- $\mathsf{tr} \leftarrow F(\mathsf{sk}, m; r)$ *for weak non-repeatability, and either*
- $\mathsf{tr} \leftarrow F(\mathsf{sk}, m; r)$ *with the nonce assumption for* r*, or*
- $\mathsf{tr} \leftarrow F(\mathsf{tsk}', m; r)$ *assuming derandomisation for strong non-repeatability.*

We provide further details and a formal proof of this theorem in Section C.

Securing Against Key Leakage. Key exposure [28,27] models full leakage of the temporary secret key tsk during the untrusted execution of DetComp, while a master secret key sk remains protected in a trusted environment. We believe key exposure fits our model best, as it is the most general form of key leakage in our setting, avoids state- or epoch-based assumptions entirely, and requires only a small number of variables. We formalize key exposure via a leakage oracle for temporary keys and the notion of unforgeable key exposure (UKE), which requires that temporary keys hide sk, are bound to the master key and the intended message, and remain publicly verifiable. Our starting point is the key-insulation paradigm of Dodis et al. [28], but their constructions rely on time periods and stateful key updates. In our generalized setting for arbitrary cryptographic algorithms, this leaves authenticating message-specific temporary keys under a master key as the natural generic approach. We therefore make this structure explicit and present a simple compiler that binds a fresh temporary verification key to the message and certifies this binding via a separate sEUF-aCMA secure signature. Building on this baseline, we further optimize within the same constraints by replacing per-message certification with a Merkle-tree approach for message sets, thereby reducing the number of trusted signatures while preserving the original security goal sg and adding protection against key exposure. Theorem 2 and Figure 3 summarize our results. Overall, this yields a generic and conceptually clean compiler for key exposure in our model, even though more specialized settings may admit further efficiency improvements.

$\mathsf{Setup}(\mathsf{sk}, \mathbb{M})$	$\mathsf{A}'_1(\mathsf{tsk}, \mathsf{vk}, H, m)$	$\mathsf{A}'_2(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma', R, \pi)$
$1: \ \textbf{if } \lvert\mathbb{M}\rvert = 0 : \textbf{return } \perp$	$1: \ \pi \leftarrow \mathsf{BuildAuthPath}(H, m)$	$1: \ b_1 \leftarrow \mathsf{VrfyPath}(R, m, \pi)$
$2: \ r \leftarrow\$ \mathcal{D}_\lambda$	$2: \ \textbf{if } \lvert\pi\rvert = 0 : \textbf{return } \perp$	$2: \ b_2 \leftarrow \mathsf{Vrfy}(\mathsf{pk}, R\|\|\mathsf{vk}, \sigma')$
$3: \ (\mathsf{tsk}, \mathsf{vk}) \leftarrow \mathsf{A}_0(r)$	$3: \ r \leftarrow\$ \mathcal{D}_R$	$3: \ b_3 \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$
$4: \ H \leftarrow \mathsf{BuildMerkleTree}(\mathbb{M})$	$4: \ \sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$	$4: \ \textbf{return } b_1 \wedge b_2 \wedge b_3$
$5: \ R \leftarrow H[0]$	$5: \ \textbf{return } (\sigma, \pi)$	
$6: \ \sigma' \leftarrow \mathsf{Sign}(\mathsf{sk}, R\|\|\mathsf{vk})$		
$7: \ \textbf{return } (\mathsf{tsk}, \mathsf{vk}, \sigma', R, H)$		

Fig. 3: Overview of the tree-based key exposure security compiler for an arbitrary cryptographic scheme $\Pi = (\mathsf{A}_0, \mathsf{A}_1, \mathsf{A}_2)$ utilizing a sEUF-aCMA secure signature scheme $\Sigma = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ and the functionalities BuildMerkleTree, BuildAuthPath, and VrfyPath for managing the Merkle-tree.

Theorem 2 (Merkle Tree Compiler). *Let $\Pi = (\mathsf{A}_0, \mathsf{A}_1, \mathsf{A}_2)$ be a cryptographic scheme w.r.t. Model 3.1 satisfying a security goal* sg *against any* PPT *adversary $\mathcal{A}$ with adaptive chosen input attack* aCMA *against algorithm A_1 and corresponding public verification algorithm A_2. Let $\Sigma = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ be a* sEUF-aCMA *secure signature scheme. Let* GenTsk *be the security compiler w.r.t. our Model in Section 3.1. Let* H *be a collision-resistant hash function. Let $\Pi' = (\mathsf{A}_0, \mathsf{A}'_1, \mathsf{A}'_2)$ be the scheme obtained from Π by applying the security compiler from Figure 3. Then, Π' satisfies* sg *with* UKE *against any* PPT *adversary $\mathcal{A}$ with* aCMA *and a key exposure oracle.*

We provide further details and a formal proof of this theorem in Section D.

4 Signature Security Framework

This section comprehensively formalizes the classical security framework by Goldwasser et al. [42] and subsequently presents the extension of the framework according to our generalized model and interface, including the security goals beyond unforgeability systematically analysed by Cremers et al. [21].

4.1 Classical Framework of Signature Security

Security Definition. The code-based game-playing framework by Bellare and Rogaway [12] defines security as a combination of a *security goal* sg (see Section A.2) with an *adversary model* am (see Definition 5), which results in our general *security definition* in Definition 3. We analyze security with a probabilistic evaluation of the cryptographic game corresponding to a security definition. This game provides the necessary administrative environment in which an adversary aims to break sg using the oracles related to am. Figure 3 presents a comprehensive and concise formalization of possible games, written in a clear and accessible style.

$(\mathsf{sg}-\mathsf{am})^{\mathcal{A}}_{\Sigma}(\lambda)$

$(\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \mathsf{Gen}(1^\lambda)$
$m^* \leftarrow \emptyset,\ \sigma^* \leftarrow \emptyset,\ \mathcal{Q} \leftarrow \emptyset,\ M \leftarrow \emptyset$
$\mathsf{Init}(\mathsf{am}, \mathsf{sg}, \lambda, \mathsf{pk})$
$\mathsf{Learn}(\mathsf{am}, \mathsf{sg}, m^*, \mathsf{sk}, \mathsf{pk})$
$\mathsf{SuccDet}(\mathsf{sg}, \mathsf{pk})$
return 0

$\mathsf{Init}(\mathsf{am}, \mathsf{sg}, \lambda, \mathsf{pk})$

$m^*, M \leftarrow\$ \mathcal{A}(1^\lambda)$
if $(\mathsf{am} = \mathrm{dCMA}) : M \leftarrow\$ \mathcal{A}(\mathsf{pk})$
if $(\mathsf{sg} = \mathrm{sSUF}) : m^* \leftarrow\$ \mathcal{A}(\mathsf{pk})$
if $(\mathsf{sg} = \mathrm{UUF}) : m^* \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M$

$\mathsf{Learn}(\mathsf{am}, \mathsf{sg}, m^*, \mathsf{sk}, \mathsf{pk})$

if $(\mathsf{sg} \notin \{\mathrm{wSUF}, \mathrm{sSUF}, \mathrm{UUF}\}) : m^* \leftarrow\$ \mathcal{A}^{\mathcal{O}^{\mathsf{am}}_{\mathsf{Sign}}(\mathsf{sk},\cdot)}(m^*, \mathsf{pk})$
$\sigma^* \leftarrow\$ \mathcal{A}^{\mathcal{O}^{\mathsf{am}}_{\mathsf{Sign}}(\mathsf{sk},\cdot)}(m^*, \mathsf{pk})$

$\mathsf{SuccDet}(\mathsf{sg}, \mathsf{pk})$

if $(\mathsf{sg} = \mathrm{UB}) : \mathsf{sk}^* \leftarrow\$ \mathcal{A}(\mathsf{pk})$
  return $\mathsf{KCheck}(\mathsf{sk}^*, \mathsf{pk})$
if $(\mathsf{sg} = \mathrm{sEUF} \wedge (m^*, \sigma^*) \notin \mathcal{Q}) \vee (m^* \notin \mathcal{Q}) :$
  return $\mathsf{Vrfy}(\mathsf{pk}, m^*, \sigma^*)$

$\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m)$

$r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_R$
$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m; r)$
$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(m, \sigma)\}$
return (m, σ)

$\mathcal{O}^{\mathrm{aCMA}}_{\mathsf{Sign}}(\mathsf{sk}, m)$

return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m)$

$\mathcal{O}^{\mathrm{dCMA}}_{\mathsf{Sign}}(\mathsf{sk}, m)$

if $M = \emptyset$: **return** $\perp$
$m \leftarrow M$
$M \leftarrow M \setminus \{m\}$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m)$

$\mathcal{O}^{\mathrm{gCMA}}_{\mathsf{Sign}}(\mathsf{sk}, m)$

return $\mathcal{O}^{\mathrm{dCMA}}_{\mathsf{Sign}}(\mathsf{sk}, m)$

$\mathcal{O}^{\mathrm{KMA}}_{\mathsf{Sign}}(\mathsf{sk}, m)$

$m \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m)$

Fig. 4: Comprehensive formalized experiment framework of digital signatures. We excluded the empty oracle $\mathcal{O}^{\mathrm{KOA}}_{\mathsf{Sign}}$. We highlighted the parts addressed by $\mathsf{sg} =$ sEUF and $\mathsf{am} =$ CMA, resulting in the sEUF-CMA security experiment. In our artefact, i.e., `https://signature-security-generator.org/` with username `reviewer` and password `access`, this corresponds to the classical framework view with security goal set to sEUF and adversary model set to aCMA.

Definition 3 (Generalized Classic Signature Security). *A signature scheme* $\Sigma = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Vrfy})$ *with key pair* $(\mathsf{sk}, \mathsf{pk})$ *achieves a* security goal sg *under a given* adversary model am*, or is* sg-am*-secure, if for all* PPT *adversaries* $\mathcal{A}$

$$\Pr[(\mathsf{sg}-\mathsf{am})^{\mathcal{A}}_{\Sigma}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$$

holds, where negl *is a* negligible function*,* $\mathsf{am} \in \{$KOA, KMA, gCMA, dCMA, aCMA$\}$*, and* $\mathsf{sg} \in \{$UB, UUF, wSUF, sSUF, wEUF, sEUF$\}$*. The probability is taken over the randomness used by* $\mathcal{A}$ *and the randomness used in the experiment.*

Experiment Formalisation. Figure 4 presents our comprehensive and redundant-free experiment formalization. We divide the formalization into two blocks. The left describes the administration of the experiment, including the formalization of

the *security goals* sg. The right defines the oracles for the various *adversary models* am. The administrative block consists of four parts. The meta part initializes keys and global variables. Subsequently, it calls three algorithms, namely *Initialization* Init, *Learning* Learn, and *Success Determination* $\mathsf{SuccDet}$. With Init, we cover the cases in which a message set M or the challenge message m^* is set in advance. In the formalization (see lines 2 and 3 of Init), we use overwriting to minimize redundancy by forcing the $\mathcal{A}$ to pre-commit data; if that commitment isn't needed, $\mathcal{A}$ can overwrite it and discard the original. To present a comprehensive formalization, we call $\mathcal{A}$ multiple times. As a result, we assume $\mathcal{A}$ to be stageful, i.e., $\mathcal{A}$ remembers the previous invocations. Learn describes the interaction of $\mathcal{A}$ with the corresponding oracle and obtains the resulting message signature pair for success determination, which $\mathsf{SuccDet}$ performs. Note that generalizing unbreakability UB is difficult without details about the relation between the keypair $(\mathsf{sk}, \mathsf{pk})$. For a candidate sk^* from $\mathcal{A}$ we cannot simply check $\mathsf{sk}^* = \mathsf{sk}$, since multiple secret keys may all verify under pk. Instead, we define $\mathsf{KCheck}(\mathsf{sk}^*, \mathsf{pk})$, which takes as input a secret key $\mathsf{sk}^* \in \mathcal{D}_\lambda^{\mathsf{sk}}$ and a public key $\mathsf{pk} \in \mathcal{D}_\lambda^{\mathsf{pk}}$, and outputs $b = 1$ if $\mathsf{Gen}(\mathsf{sk}^*) = \mathsf{pk}$, and $b = 0$ otherwise. Here, $\mathsf{Gen}\colon \mathcal{D}_\lambda^{\mathsf{sk}} \to \mathcal{D}_\lambda^{\mathsf{pk}}$ denotes a subprocedure of the key generation that deterministically outputs $\mathsf{Gen}(\mathsf{sk}) = \mathsf{pk}$ for any input secret key sk. The oracles in step Learn may need sampling (see line 1 of $\mathcal{O}_{\mathsf{Sign}}$ or $\mathcal{O}_{\mathsf{Sign}}^{\mathrm{KMA}}$), not only for randomness values but also for messages. We introduce $\mathcal{D}_M^{\mathsf{s}}$ and $\mathcal{D}_R^{\mathsf{s}}$, a space of messages similar to $\mathcal{D}_M$ and randomness $\mathcal{D}_R$ but with a specific probability distribution, respectively. Note that a concrete construction needs to instantiate these distributions. Usually, $\mathcal{D}_R^{\mathsf{s}}$ corresponds to the uniform distribution. Further, we divide the oracle block according to the individual adversary models and create an oracle for each adversary model, identifiable by the superscript in Learn. Oracle $\mathcal{O}_{\mathsf{Sign}}$ acts as a default oracle, which is eventually called by all other oracles to reduce redundancy. Additionally, $\mathcal{O}_{\mathsf{Sign}}$ manages the global variable $\mathcal{Q}$, which is necessary for the administrative block and contains all message signature pairs received from $\mathcal{A}$. For clarification, we define $\mathcal{Q} = \{(\mathcal{Q}_{\mathsf{Sign}}^m[k], \mathcal{Q}_{\mathsf{Sign}}^\sigma[k]\}_k$ with reference to Section 3.2. Our formalization covers the classical signature security model while avoiding additional cryptographic concepts to maintain focus. Customization is possible by simply adding elements, such as another oracle $\mathcal{O}'(\cdot)$ in line 1 of Learn in Figure 4, resulting in **if** $(\mathsf{sg} \notin \{\mathrm{wSUF}, \mathrm{sSUF}, \mathrm{UUF}\})$: $m^* \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Sign}}^{\mathsf{am}}(\mathsf{sk},\cdot), \mathcal{O}'(\cdot)}(m^*, \mathsf{pk})$.

4.2 Extended Security Definition for Signatures

We now extend the formalization of Section 4.1 with the security goals beyond unforgeability by Cremers et al. [21] and the attack vectors from Section 3.2. Our approach combines, generalizes, and extends beyond the current literature. Since the resulting formalization is dense and difficult to parse, we refer the reader to our artifact, available at `https://signature-security-generator.org/` with username `reviewer` and password `access`, which presents the corresponding definitions, experiments, and oracles in a compact and interactive way.

$(\mathsf{sg}-\mathsf{am})^{\mathcal{A}}_{\Sigma}(\lambda)$

$(\mathsf{sk},\mathsf{pk}) \leftarrow\$ \mathsf{Gen}(1^\lambda), \mathcal{Q} \leftarrow \emptyset,\ \mathcal{Q}_l \leftarrow \emptyset,\ M \leftarrow \emptyset,\ R \leftarrow \emptyset$

$\mathsf{Init}(\mathsf{am},\mathsf{sg},\mathsf{sk},\lambda,\mathsf{pk});\quad \mathsf{Learn}(\mathsf{am},\mathsf{sg},\mathsf{sk},\mathsf{pk});\quad \mathsf{SuccDet}(\mathsf{sg},\mathsf{pk});\quad \textbf{return } 0$

$\mathsf{Init}(\mathsf{am},\mathsf{sg},\mathsf{sk},\lambda,\mathsf{pk})$

$m^*, M, R \leftarrow\$ \mathcal{A}^{\mathsf{am}}_{\mathsf{sg}}(\lambda)$

$\textbf{if } (\mathsf{mc}=\mathrm{dCMA}) : M \leftarrow\$ \mathcal{A}(\mathsf{pk});\ \textbf{if } (\mathsf{rc}=\mathrm{dCRA}) : R \leftarrow\$ \mathcal{A}(\mathsf{pk})$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{sSUF}\} \neq \emptyset) : m^* \leftarrow\$ \mathcal{A}(\mathsf{pk});\ \textbf{if } (\mathsf{sg} \cap \{\mathrm{UUF}\} \neq \emptyset) : m^* \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{NR}\} \neq \emptyset) : (m^*_R, \mathsf{aux}) \leftarrow\$ \mathcal{D}(\lambda,\mathsf{pk}),\ r \leftarrow\$ \mathcal{D}_R,\ \sigma^*_R \leftarrow \mathsf{Sign}(\mathsf{sk}, m^*_R; r)$

$\mathsf{Learn}(\mathsf{am},\mathsf{sg},\mathsf{sk},\mathsf{pk})$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{wSUF},\mathrm{sSUF},\mathrm{UUF}\} = \emptyset) :\ m^* \leftarrow\$ \mathcal{A}^{\mathcal{O}^{(\mathsf{mc},\mathsf{rc})}_{\mathsf{Sign}}(\mathsf{sk},\cdot;\cdot),\mathcal{O}^{L}_{\mathsf{Sign}}(\cdot,\cdot)}(m^*, \sigma^*_R, \mathsf{aux}, \mathsf{pk})$

$(\mathsf{sk}^*, \mathsf{pk}^*, \sigma^*) \leftarrow\$ \mathcal{A}^{\mathcal{O}^{(\mathsf{mc},\mathsf{rc})}_{\mathsf{Sign}}(\mathsf{sk},\cdot;\cdot),\mathcal{O}^{L}_{\mathsf{Sign}}(\cdot,\cdot)}(m^*, \sigma^*_R, \mathsf{aux}, \mathsf{pk})$

$\mathsf{SuccDet}(\mathsf{sg},\mathsf{pk})$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{UB}\} \neq \emptyset \wedge \mathsf{KCheck}(\mathsf{sk}^*,\mathsf{pk})) : \textbf{return } 1$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{UKE}\} \neq \emptyset \wedge (m^* \notin \mathcal{Q}_l)) : \textbf{return } 0$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{sEUF}\} \neq \emptyset \wedge (m^*,\sigma^*) \notin \mathcal{Q} \wedge \mathsf{Vrfy}(\mathsf{pk}, m^*, \sigma^*)) : \textbf{return } 1$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{UUF},\mathrm{wSUF},\mathrm{sSUF},\mathrm{wEUF}\} \neq \emptyset \wedge m^* \notin \mathcal{Q} \wedge \mathsf{Vrfy}(\mathsf{pk}, m^*, \sigma^*)) : \textbf{return } 1$

$b \leftarrow 1$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{wDEO},\mathrm{wCEO},\mathrm{wUEO}\} \neq \emptyset) : b \leftarrow \mathsf{KCheck}(\mathsf{sk}^*, \mathsf{pk}^*)$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{wDEO},\mathrm{sDEO}\} \neq \emptyset \wedge \mathsf{pk}^* \neq \mathsf{pk} \wedge \sigma^* \in \mathcal{Q} \wedge (m^*,\sigma^*) \notin \mathcal{Q} \wedge \mathsf{Vrfy}(\mathsf{pk}^*, m^*, \sigma^*) \wedge b) :$

$\quad \textbf{return } 1$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{wCEO},\mathrm{sCEO}\} \neq \emptyset \wedge \mathsf{pk}^* \neq \mathsf{pk} \wedge (m^*,\sigma^*) \in \mathcal{Q} \wedge \mathsf{Vrfy}(\mathsf{pk}^*, m^*, \sigma^*) \wedge b) : \textbf{return } 1$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{wUEO},\mathrm{sUEO}\} \neq \emptyset \wedge \mathsf{pk}^* \neq \mathsf{pk} \wedge \sigma^* \in \mathcal{Q} \wedge \mathsf{Vrfy}(\mathsf{pk}^*, m^*, \sigma^*) \wedge b) : \textbf{return } 1$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{NR}\} \neq \emptyset \wedge \mathsf{pk} \neq \mathsf{pk}^* \wedge \mathsf{Vrfy}(\mathsf{pk}^*, m^*_R, \sigma^*)) : \textbf{return } 1$

$m^*_1 || m^*_2 \leftarrow m^*$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{MB}\} \neq \emptyset \wedge m^*_1 \neq m^*_2 \wedge \mathsf{Vrfy}(\mathsf{pk}^*, m^*_1, \sigma^*) \wedge \mathsf{Vrfy}(\mathsf{pk}^*, m^*_2, \sigma^*)) : \textbf{return } 1$

$\mathsf{pk}^*_1 || \mathsf{pk}^*_2 \leftarrow \mathsf{pk}^*$

$\textbf{if } (\mathsf{sg} \cap \{\mathrm{msUEO}\} \neq \emptyset \wedge \mathsf{pk}^*_1 \neq \mathsf{pk}^*_2 \wedge \mathsf{Vrfy}(\mathsf{pk}^*_1, m^*_1, \sigma^*) \wedge \mathsf{Vrfy}(\mathsf{pk}^*_2, m^*_2, \sigma^*)) : \textbf{return } 1$

Fig. 5: Extended formalization of the administrative part of Figure 4.

Definition 4 (Generalized Security for Digital Signatures). *A signature scheme* $\Sigma = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Vrfy})$ *with key pair* $(\mathsf{sk}, \mathsf{pk})$ *achieves a* security goal sg *under a given* adversary model $\mathsf{am} = (\mathsf{mc}, \mathsf{rc}, \mathbb{L})$ *against* Sign *with the assumption of security for* KGen *and* Vrfy*, if for all* PPT *adversaries* $\mathcal{A}$ *it holds*

$$\Pr[(\mathsf{sg}-\mathsf{am})^{\mathcal{A}}_{\Sigma}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$$

where negl *is a* negligible function*,* $\mathcal{D}$ *is a* PPT *sampler with computationally independent auxiliary data* aux *for* KGen *and conditional (HILL) min-entropy of output* (m, aux) *strictly greater than logarithmic in* λ*, with security goals* $\mathsf{sg} \in \{\mathrm{UB}, \mathrm{UUF}, \mathrm{wSUF}, \mathrm{sSUF}, \mathrm{wEUF}, \mathrm{sEUF}, \mathrm{wDEO}, \mathrm{sDEO}, \mathrm{wCEO}, \mathrm{sCEO}, \mathrm{wUEO}, \mathrm{sUEO}, \mathrm{msUEO}, \mathrm{UKE}, \mathrm{MB}, \mathrm{NR}\}$*, message choice* $\mathsf{mc} \in \{\mathrm{KO}, \mathrm{KMA}, \mathrm{gCMA}, \mathrm{dCMA}, \mathrm{aCMA}\}$*, randomness choice* $\mathsf{rc} \in \{\mathrm{KO}, \mathrm{KRA}, \mathrm{gCRA}, \mathrm{dCRA}, \mathrm{aCRA}\}$*, and*

$\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathsf{tr} \leftarrow \mathsf{GenTr}(\mathsf{sk}, \mathsf{tsk}, m; r)$
$\mathsf{tsk} \leftarrow \mathsf{GenTsk}(\mathsf{sk}, \mathsf{tr})$
$\sigma \leftarrow \mathsf{Sign}(\mathsf{tsk}, m; \mathsf{tr})$
$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(m, \sigma, \mathsf{tr}, \mathsf{tsk})\}$
return σ

Mixed Without Direct Choice:

$\mathcal{O}^{(\mathrm{aCMA,aCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{aCMA,KRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_R$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r), r$

$\mathcal{O}^{(\mathrm{KMA,aCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$m \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r), m$

$\mathcal{O}^{(\mathrm{KMA,KRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$m,\ r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M,\ \mathcal{D}^{\mathsf{s}}_R$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r), m, r$

Key Exposure Oracle:

$\mathcal{O}^{L}_{\mathsf{Sign}}(m, \sigma)$

$t = \{(m', \sigma', \cdot, \mathsf{tsk}') \in \mathcal{Q}$
$\quad \mid m' = m \wedge \sigma' = \sigma\}$
if $(t \neq \emptyset) : \mathcal{Q}_l \leftarrow \mathcal{Q}_l \cup \{m\}$
$\mathsf{tsk}' \leftarrow t$
return tsk'

Message Only Attacks:

$\mathcal{O}^{(\mathrm{aCMA,KO})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_R$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{dCMA,KO})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

if $M = \emptyset$: **return** $\perp$
$m,\ M \leftarrow M,\ M \setminus \{m\}$
$r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_R$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{KMA,KO})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$m,\ r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M,\ \mathcal{D}^{\mathsf{s}}_R$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r), m$

Randomness Only Attacks:

$\mathcal{O}^{(\mathrm{KO,aCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$m \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{KO,dCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

if $R = \emptyset$: **return** $\perp$
$r,\ R \leftarrow R,\ R \setminus \{r\}$
$m \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{KO,KRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$m,\ r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_M,\ \mathcal{D}^{\mathsf{s}}_R$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r), r$

Mixed With Direct Choice:

$\mathcal{O}^{(\mathrm{aCMA,dCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

if $R = \emptyset$: **return** $\perp$
$r,\ R \leftarrow R,\ R \setminus \{r\}$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{dCMA,aCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

if $M = \emptyset$: **return** $\perp$
$m,\ M \leftarrow M,\ M \setminus \{m\}$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{dCMA,dCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

if $M = \emptyset$ or $R = \emptyset$:
$\quad$ **return** $\perp$
$m,\ r \leftarrow M,\ R$
$M, R \leftarrow M \setminus \{m\}, R \setminus \{r\}$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

$\mathcal{O}^{(\mathrm{dCMA,KRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

if $M = \emptyset$: **return** $\perp$
$m,\ M \leftarrow M,\ M \setminus \{m\}$
$r \leftarrow\$ \mathcal{D}^{\mathsf{s}}_R$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r), r$

$\mathcal{O}^{(\mathrm{KMA,dCRA})}_{\mathsf{Sign}}(\mathsf{sk}, m; r)$

if $R = \emptyset$: **return** $\perp$
$r,\ R \leftarrow R,\ R \setminus \{r\}$
$m \leftarrow \mathcal{D}^{\mathsf{s}}_M$
return $\mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, m; r), m$

Fig. 6: Extended formalization of the oracle part of Figure 4.

leakage function(s) $\mathbb{L} = \{L^{m,\sigma}_{\mathsf{tsk}}\}$. *The probability is taken over the randomness used by* $\mathcal{A}$ *and* $\mathcal{D}$ *and the randomness used in the experiment.*

Formalization. In the administrative part (see Figure 5), we assume sg is a (non-empty) set of security goals and the adversary wins if he can break one of these goals. Further, we assume the adversary has one unforgeability goal and one exclusive ownership goal, since combining several goals of the same class is not meaningful. We include the security goal unforgeable key exposure UKE from Section D, which states that the (temporary) secret key for the final (challenge) output is not leaked. We identify the oracles with $\mathcal{O}^{(\mathsf{mc},\mathsf{rc})}_{\mathsf{Sign}}(\mathsf{sk}, \cdot; \cdot)$, with message choice mc and randomness choice rc. We divide the oracle formalization (see Figure 6) into six parts. Oracle $\mathcal{O}_{\mathsf{Sign}}$ serves as the basis. Regarding our model (cf. Figure 2), this includes the modifications of the randomness r with GenTr to create the internal randomness tr and similarly GenTsk to modify sk to create tsk

(see lines 1 and 2 of $\mathcal{O}_{\mathsf{Sign}}$ in Figure 6). We have four conceptual categories for the oracles with message and/or randomness choice, namely message-only attacks, randomness-only attacks, mixed oracles without direct choice, and mixed oracles with direct choice. In contrast to the message and randomness oracles, we can formalize key exposure as an independent additional oracle $\mathcal{O}_{\mathsf{Sign}}^{L_{\mathsf{tsk}}^{m,\sigma}}(m,\sigma)$ with simplified notation $\mathcal{O}_{\mathsf{Sign}}^{L}(m,\sigma)$. Note that we do not include the oracles with a generic choice. These oracles only differ from those with direct choices in the Init part of Figure 5. Consequently, the oracles for $(\mathrm{gCMA}, \mathrm{KO})$ calls the oracle for $(\mathrm{dCMA}, \mathrm{KO})$ and $(\mathrm{KO}, \mathrm{gCRA})$ calls $(\mathrm{KO}, \mathrm{dCRA})$. For an arbitrary message choice mc the oracles for $(\mathsf{mc}, \mathrm{gCRA})$ call the oracles for $(\mathsf{mc}, \mathrm{dCRA})$ and similarly for an arbitrary randomness choice rc oracles $(\mathrm{gCMA}, \mathsf{rc})$ call $(\mathrm{dCMA}, \mathsf{rc})$.

A Broader Perspective. In Section 3, we switch our perspective from signatures to a generalized model for cryptographic schemes, which we call the "security interface". This enables a systematic description of potential adversary models and security definitions, including those beyond message attacks, within a unified model. Moreover, the abstraction enables modularity: it isolates the relevant interfaces and assumptions in a way that allows our results, including the security compilers from Section 3.3, to transfer naturally to other cryptographic schemes that fit the model. This formalization section instantiates the generalized interface to provide a "signature security framework" that combines our adversary models, security compilers, and recent advances in security goals into a comprehensive and up-to-date view of signature security. As a practical application of our formalization, the accompanying code artifact turns the resulting definitions, experiments, and oracles into a compact and interactive representation, making the framework substantially easier to explore and use in practice. Finally, in Section E, we show how to directly apply our results to an existing standardized signature scheme by combining the security compilers with the security framework in the case of the Schnorr signature scheme [71].

References

1. Abdalla, M., Reyzin, L.: A new forward-secure digital signature scheme. In: Okamoto, T. (ed.) Advances in Cryptology – ASIACRYPT 2000. Lecture Notes in Computer Science, vol. 1976, pp. 116–129. Springer Berlin Heidelberg, Germany, Kyoto, Japan (Dec 3–7, 2000). `https://doi.org/10.1007/3-540-44448-3_10`
2. Alekseev, E.K., Akhmetzyanova, L., Babueva, A.A., Smyshlyaev, S.V.: Improving the security of elgamal-type signatures. Matematicheskie Voprosy Kriptografii [Mathematical Aspects of Cryptography] **12**(3), 5–30 (2021)
3. Alwen, J., Dodis, Y., Wichs, D.: Leakage-resilient public-key cryptography in the bounded-retrieval model. In: Halevi, S. (ed.) Advances in Cryptology – CRYPTO 2009. Lecture Notes in Computer Science, vol. 5677, pp. 36–54. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 16–20, 2009). `https://doi.org/10.1007/978-3-642-03356-8_3`
4. An, J.H., Dodis, Y., Rabin, T.: On the security of joint signature and encryption. In: International conference on the theory and applications of cryptographic techniques. pp. 83–107. Springer (2002)

5. Aranha, D.F., Orlandi, C., Takahashi, A., Zaverucha, G.: Security of hedged Fiat-Shamir signatures under fault attacks. In: Canteaut, A., Ishai, Y. (eds.) Advances in Cryptology – EUROCRYPT 2020, Part I. Lecture Notes in Computer Science, vol. 12105, pp. 644–674. Springer, Cham, Switzerland, Zagreb, Croatia (May 10–14, 2020). https://doi.org/10.1007/978-3-030-45721-1_23
6. Baek, J., Kim, K.: Remarks on the unknown key share attacks. IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences **83**(12), 2766–2769 (2000)
7. Barenghi, A., Breveglieri, L., Koren, I., Naccache, D.: Fault injection attacks on cryptographic devices: Theory, practice, and countermeasures. Proceedings of the IEEE **100**(11), 3056–3076 (2012). https://doi.org/10.1109/JPROC.2012.2188769
8. Bellare, M., Goldwasser, S.: Verifiable partial key escrow. In: Graveman, R., Janson, P.A., Neuman, C., Gong, L. (eds.) ACM CCS 97: 4th Conference on Computer and Communications Security. pp. 78–91. ACM Press, Zurich, Switzerland (Apr 1–4, 1997). https://doi.org/10.1145/266420.266439
9. Bellare, M., Micciancio, D., Warinschi, B.: Foundations of group signatures: Formal definitions, simplified requirements, and a construction based on general assumptions. In: Biham, E. (ed.) Advances in Cryptology – EUROCRYPT 2003. Lecture Notes in Computer Science, vol. 2656, pp. 614–629. Springer Berlin Heidelberg, Germany, Warsaw, Poland (May 4–8, 2003). https://doi.org/10.1007/3-540-39200-9_38
10. Bellare, M., Miner, S.K.: A forward-secure digital signature scheme. In: Wiener, M.J. (ed.) Advances in Cryptology – CRYPTO'99. Lecture Notes in Computer Science, vol. 1666, pp. 431–448. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 15–19, 1999). https://doi.org/10.1007/3-540-48405-1_28
11. Bellare, M., Namprempre, C., Neven, G.: Security proofs for identity-based identification and signature schemes. Journal of Cryptology **22**(1), 1–61 (Jan 2009). https://doi.org/10.1007/s00145-008-9028-8
12. Bellare, M., Rogaway, P.: The security of triple encryption and a framework for code-based game-playing proofs. In: Vaudenay, S. (ed.) Advances in Cryptology – EUROCRYPT 2006. Lecture Notes in Computer Science, vol. 4004, pp. 409–426. Springer Berlin Heidelberg, Germany, St. Petersburg, Russia (May 28 – Jun 1, 2006). https://doi.org/10.1007/11761679_25
13. Bellare, M., Tackmann, B.: Nonce-based cryptography: Retaining security when randomness fails. In: Fischlin, M., Coron, J.S. (eds.) Advances in Cryptology – EUROCRYPT 2016, Part I. Lecture Notes in Computer Science, vol. 9665, pp. 729–757. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). https://doi.org/10.1007/978-3-662-49890-3_28
14. Blake-Wilson, S., Menezes, A.: Unknown key-share attacks on the station-to-station (STS) protocol. In: Imai, H., Zheng, Y. (eds.) PKC'99: 2nd International Workshop on Theory and Practice in Public Key Cryptography. Lecture Notes in Computer Science, vol. 1560, pp. 154–170. Springer Berlin Heidelberg, Germany, Kamakura, Japan (Mar 1–3, 1999). https://doi.org/10.1007/3-540-49162-7_12
15. Bohli, J.M., Röhrich, S., Steinwandt, R.: Key substitution attacks revisited: Taking into account malicious signers. International Journal of Information Security **5**(1), 30–36 (2006)
16. Boyle, E., Segev, G., Wichs, D.: Fully leakage-resilient signatures. Journal of Cryptology **26**(3), 513–558 (Jul 2013). https://doi.org/10.1007/s00145-012-9136-3

17. Brendel, J., Cremers, C., Jackson, D., Zhao, M.: The provable security of Ed25519: Theory and practice. In: 2021 IEEE Symposium on Security and Privacy. pp. 1659–1676. IEEE Computer Society Press, San Francisco, CA, USA (May 24–27, 2021). https://doi.org/10.1109/SP40001.2021.00042
18. Brown, D.R.: A weak-randomizer attack on rsa-oaep with e= 3. Cryptology ePrint Archive (2005)
19. Camenisch, J., Lysyanskaya, A.: A signature scheme with efficient protocols. In: Cimato, S., Galdi, C., Persiano, G. (eds.) SCN 02: 3rd International Conference on Security in Communication Networks. Lecture Notes in Computer Science, vol. 2576, pp. 268–289. Springer Berlin Heidelberg, Germany, Amalfi, Italy (Sep 12–13, 2003). https://doi.org/10.1007/3-540-36413-7_20
20. Checkoway, S., Niederhagen, R., Everspaugh, A., Green, M., Lange, T., Ristenpart, T., Bernstein, D.J., Maskiewicz, J., Shacham, H., Fredrikson, M.: On the practical exploitability of dual EC in TLS implementations. In: Fu, K., Jung, J. (eds.) USENIX Security 2014: 23rd USENIX Security Symposium. pp. 319–335. USENIX Association, San Diego, CA, USA (Aug 20–22, 2014), https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/checkoway
21. Cremers, C., Düzlü, S., Fiedler, R., Fischlin, M., Janson, C.: BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures. In: 2021 IEEE Symposium on Security and Privacy. pp. 1696–1714. IEEE Computer Society Press, San Francisco, CA, USA (May 24–27, 2021). https://doi.org/10.1109/SP40001.2021.00093
22. Davis, H., Green, M.D., Heninger, N., Ryan, K., Suhl, A.: On the possibility of a backdoor in the Micali-Schnorr generator. In: Tang, Q., Teague, V. (eds.) PKC 2024: 27th International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science, vol. 14601, pp. 352–386. Springer, Cham, Switzerland, Sydney, NSW, Australia (Apr 15–17, 2024). https://doi.org/10.1007/978-3-031-57718-5_12
23. Degabriele, J.P., Paterson, K.G., Schuldt, J.C.N., Woodage, J.: Backdoors in pseudorandom number generators: Possibility and impossibility results. In: Robshaw, M., Katz, J. (eds.) Advances in Cryptology – CRYPTO 2016, Part I. Lecture Notes in Computer Science, vol. 9814, pp. 403–432. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 14–18, 2016). https://doi.org/10.1007/978-3-662-53018-4_15
24. Diffie, W., Hellman, M.E.: New directions in cryptography. In: Secure communications and asymmetric cryptosystems, pp. 143–180. Routledge (2019)
25. Ding, J., Schmidt, D.: Rainbow, a new multivariable polynomial signature scheme. In: Ioannidis, J., Keromytis, A., Yung, M. (eds.) ACNS 2005: 3rd International Conference on Applied Cryptography and Network Security. Lecture Notes in Computer Science, vol. 3531, pp. 164–175. Springer Berlin Heidelberg, Germany, New York, NY, USA (Jun 7–10, 2005). https://doi.org/10.1007/11496137_12
26. Dodis, Y., Haralambiev, K., López-Alt, A., Wichs, D.: Efficient public-key cryptography in the presence of key leakage. In: Abe, M. (ed.) Advances in Cryptology – ASIACRYPT 2010. Lecture Notes in Computer Science, vol. 6477, pp. 613–631. Springer Berlin Heidelberg, Germany, Singapore (Dec 5–9, 2010). https://doi.org/10.1007/978-3-642-17373-8_35
27. Dodis, Y., Katz, J., Xu, S., Yung, M.: Key-insulated public key cryptosystems. In: Knudsen, L.R. (ed.) Advances in Cryptology – EUROCRYPT 2002. Lecture Notes in Computer Science, vol. 2332, pp. 65–82. Springer Berlin Heidelberg, Germany,

Amsterdam, The Netherlands (Apr 28 – May 2, 2002). https://doi.org/10.1007/3-540-46035-7_5
28. Dodis, Y., Katz, J., Xu, S., Yung, M.: Strong key-insulated signature schemes. In: Desmedt, Y. (ed.) PKC 2003: 6th International Workshop on Theory and Practice in Public Key Cryptography. Lecture Notes in Computer Science, vol. 2567, pp. 130–144. Springer Berlin Heidelberg, Germany, Miami, FL, USA (Jan 6–8, 2003). https://doi.org/10.1007/3-540-36288-6_10
29. Dodis, Y., Ong, S.J., Prabhakaran, M., Sahai, A.: On the (im)possibility of cryptography with imperfect randomness. In: 45th Annual Symposium on Foundations of Computer Science. pp. 196–205. IEEE Computer Society Press, Rome, Italy (Oct 17–19, 2004). https://doi.org/10.1109/FOCS.2004.44
30. Dolev, D., Dwork, C., Naor, M.: Non-malleable cryptography. In: Proceedings of the Twenty-Third Annual ACM Symposium on Theory of Computing. pp. 542–552. STOC '91, Association for Computing Machinery, New York, NY, USA (1991). https://doi.org/10.1145/103418.103474, https://doi.org/10.1145/103418.103474
31. Don, J., Fehr, S., Huang, Y.H., Struck, P.: On the (in)security of the BUFF transform. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part I. Lecture Notes in Computer Science, vol. 14920, pp. 246–275. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68376-3_8
32. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Dilithium: A lattice-based digital signature scheme. IACR Transactions on Cryptographic Hardware and Embedded Systems **2018**(1), 238–268 (2018). https://doi.org/10.13154/tches.v2018.i1.238-268, https://tches.iacr.org/index.php/TCHES/article/view/839
33. Dumezy, J.: Securitynotionszoo. https://github.com/jdumezy/securitynotionszoo (2026)
34. Dziembowski, S., Pietrzak, K.: Leakage-resilient cryptography. In: 49th Annual Symposium on Foundations of Computer Science. pp. 293–302. IEEE Computer Society Press, Philadelphia, PA, USA (Oct 25–28, 2008). https://doi.org/10.1109/FOCS.2008.56
35. Easttom, C.: A study of cryptographic backdoors in cryptographic primitives. In: Electrical Engineering (ICEE), Iranian Conference on. pp. 1664–1669. IEEE (2018)
36. Faust, S., Kiltz, E., Pietrzak, K., Rothblum, G.N.: Leakage-resilient signatures. In: Micciancio, D. (ed.) TCC 2010: 7th Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 5978, pp. 343–360. Springer Berlin Heidelberg, Germany, Zurich, Switzerland (Feb 9–11, 2010). https://doi.org/10.1007/978-3-642-11799-2_21
37. Fischlin, M., Günther, F.: Modeling memory faults in signature and authenticated encryption schemes. In: Jarecki, S. (ed.) Topics in Cryptology – CT-RSA 2020. Lecture Notes in Computer Science, vol. 12006, pp. 56–84. Springer, Cham, Switzerland, San Francisco, CA, USA (Feb 24–28, 2020). https://doi.org/10.1007/978-3-030-40186-3_4
38. Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z., et al.: Falcon: Fast-fourier lattice-based compact signatures over ntru. Submission to the NIST's post-quantum cryptography standardization process **36**(5), 1–75 (2018)
39. Giraud, C., Knudsen, E.W.: Fault attacks on signature schemes. In: Australasian Conference on Information Security and Privacy. pp. 478–491. Springer (2004)

40. Goldberg, I., Wagner, D.: Randomness and the netscape browser. Dr Dobb's Journal-Software Tools for the Professional Programmer **21**(1), 66–71 (1996)
41. Goldwasser, S., Micali, S., Rivest, R.L.: A "paradoxical" solution to the signature problem (extended abstract). In: 25th Annual Symposium on Foundations of Computer Science. pp. 441–448. IEEE Computer Society Press, Singer Island, Florida (Oct 24–26, 1984). https://doi.org/10.1109/SFCS.1984.715946
42. Goldwasser, S., Micali, S., Rivest, R.L.: A digital signature scheme secure against adaptive chosen-message attacks. SIAM Journal on computing **17**(2), 281–308 (1988)
43. Gordon, S.D., Katz, J., Vaikuntanathan, V.: A group signature scheme from lattice assumptions. In: Abe, M. (ed.) Advances in Cryptology – ASIACRYPT 2010. Lecture Notes in Computer Science, vol. 6477, pp. 395–412. Springer Berlin Heidelberg, Germany, Singapore (Dec 5–9, 2010). https://doi.org/10.1007/978-3-642-17373-8_23
44. Guo, C., Pereira, O., Peters, T., Standaert, F.X.: Authenticated encryption with nonce misuse and physical leakage: Definitions, separation results and first construction - (extended abstract). In: Schwabe, P., Thériault, N. (eds.) Progress in Cryptology - LATINCRYPT 2019: 6th International Conference on Cryptology and Information Security in Latin America. Lecture Notes in Computer Science, vol. 11774, pp. 150–172. Springer, Cham, Switzerland, Santiago, Chile (Oct 2–4, 2019). https://doi.org/10.1007/978-3-030-30530-7_8
45. Hanzlik, L., Kluczniak, K., Kutylowski, M.: Controlled randomness–a defense against backdoors in cryptographic devices. In: International Conference on Cryptology in Malaysia. pp. 215–232. Springer (2016)
46. Hutchinson, D.: Randomness in cryptography: theory meets practice. Ph.D. thesis, Royal Holloway, University of London (2019)
47. Jackson, D., Cremers, C., Cohn-Gordon, K., Sasse, R.: Seems legit: Automated analysis of subtle attacks on protocols that use signatures. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019: 26th Conference on Computer and Communications Security. pp. 2165–2180. ACM Press, London, UK (Nov 11–15, 2019). https://doi.org/10.1145/3319535.3339813
48. Joye, M., Tunstall, M., et al.: Fault analysis in cryptography, vol. 147. Springer (2012)
49. Katz, J., Lindell, Y.: Introduction to modern cryptography. CRC press (2020)
50. Katz, J., Vaikuntanathan, V.: Signature schemes with bounded leakage resilience. In: Matsui, M. (ed.) Advances in Cryptology – ASIACRYPT 2009. Lecture Notes in Computer Science, vol. 5912, pp. 703–720. Springer Berlin Heidelberg, Germany, Tokyo, Japan (Dec 6–10, 2009). https://doi.org/10.1007/978-3-642-10366-7_41
51. Katz, J., Wang, N.: Efficiency improvements for signature schemes with tight security reductions. In: Jajodia, S., Atluri, V., Jaeger, T. (eds.) ACM CCS 2003: 10th Conference on Computer and Communications Security. pp. 155–164. ACM Press, Washington, DC, USA (Oct 27–30, 2003). https://doi.org/10.1145/948109.948132
52. Krzywiecki, L.: Schnorr-like identification scheme resistant to malicious subliminal setting of ephemeral secret. In: International Conference for Information Technology and Communications. pp. 137–148. Springer (2016)
53. Krzywiecki, L., Koziel, P., Panwar, N.: Signature based authentication for ephemeral setup attacks in vehicular sensor networks. In: 2019 IEEE 18th International Symposium on Network Computing and Applications (NCA). pp. 1–4. IEEE (2019)

54. Krzywiecki, L., Kutylowski, M., Rothenberger, R., Drzazga, B.: Hierarchical ring signatures immune to randomness injection attacks. In: International Symposium on Cyber Security Cryptography and Machine Learning. pp. 171–186. Springer (2021)
55. Krzywiecki, L., Polubek, A., Salin, H.: Multi-signature scheme resistant to randomness injection attacks-a bitcoin case. In: 2021 IEEE 20th International Symposium on Network Computing and Applications (NCA). pp. 1–4. IEEE (2021)
56. Krzywiecki, L., Salin, H., Panwar, N., Pavlov, M.: Proxy signcryption scheme for vehicle infrastructure immune to randomness leakage and setup attacks. In: 2020 IEEE 19th International Symposium on Network Computing and Applications (NCA). pp. 1–8. IEEE (2020)
57. Krzywiecki, L., Slowik, M., Szala, M.: Identity-based signature scheme secure in ephemeral setup and leakage scenarios. In: International Conference on Information Security Practice and Experience. pp. 310–324. Springer (2019)
58. LaMacchia, B.A., Lauter, K., Mityagin, A.: Stronger security of authenticated key exchange. In: Susilo, W., Liu, J.K., Mu, Y. (eds.) ProvSec 2007: 1st International Conference on Provable Security. Lecture Notes in Computer Science, vol. 4784, pp. 1–16. Springer Berlin Heidelberg, Germany, Wollongong, Australia (Nov 1–2, 2007). https://doi.org/10.1007/978-3-540-75670-5_1
59. Lysyanskaya, A., Micali, S., Reyzin, L., Shacham, H.: Sequential aggregate signatures from trapdoor permutations. In: Cachin, C., Camenisch, J. (eds.) Advances in Cryptology – EUROCRYPT 2004. Lecture Notes in Computer Science, vol. 3027, pp. 74–90. Springer Berlin Heidelberg, Germany, Interlaken, Switzerland (May 2–6, 2004). https://doi.org/10.1007/978-3-540-24676-3_5
60. Malkin, T., Teranishi, I., Vahlis, Y., Yung, M.: Signatures resilient to continual leakage on memory and computation. In: Ishai, Y. (ed.) TCC 2011: 8th Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 6597, pp. 89–106. Springer Berlin Heidelberg, Germany, Providence, RI, USA (Mar 28–30, 2011). https://doi.org/10.1007/978-3-642-19571-6_7
61. Menezes, A., Smart, N.P.: Security of signature schemes in a multi-user setting. Designs, Codes and Cryptography **33**(3), 261–274 (2004). https://doi.org/10.1023/B:DESI.0000036250.18062.3f
62. Merkle, R.C.: A digital signature based on a conventional encryption function. In: Conference on the theory and application of cryptographic techniques. pp. 369–378. Springer (1987)
63. Naor, M., Segev, G.: Public-key cryptosystems resilient to key leakage. In: Halevi, S. (ed.) Advances in Cryptology – CRYPTO 2009. Lecture Notes in Computer Science, vol. 5677, pp. 18–35. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 16–20, 2009). https://doi.org/10.1007/978-3-642-03356-8_2
64. Ohtake, G., Hanaoka, G., Ogawa, K.: An efficient strong key-insulated signature scheme and its application. In: Public Key Infrastructure: 5th European PKI Workshop: Theory and Practice, EuroPKI 2008 Trondheim, Norway, June 16-17, 2008 Proceedings 5. pp. 150–165. Springer (2008)
65. Okamoto, T.: Provably secure and practical identification schemes and corresponding signature schemes. In: Brickell, E.F. (ed.) Advances in Cryptology – CRYPTO'92. Lecture Notes in Computer Science, vol. 740, pp. 31–53. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 16–20, 1993). https://doi.org/10.1007/3-540-48071-4_3
66. Pereira, O., Standaert, F.X., Vivek, S.: Leakage-resilient authentication and encryption from symmetric cryptographic primitives. In: Ray, I., Li, N., Kruegel,

C. (eds.) ACM CCS 2015: 22nd Conference on Computer and Communications Security. pp. 96–108. ACM Press, Denver, CO, USA (Oct 12–16, 2015). https://doi.org/10.1145/2810103.2813626
67. Poddebniak, D., Somorovsky, J., Schinzel, S., Lochter, M., Rösler, P.: Attacking deterministic signature schemes using fault attacks. In: 2018 IEEE European Symposium on Security and Privacy. pp. 338–352. IEEE Computer Society Press, London, United Kingdom (Apr 24–26, 2018). https://doi.org/10.1109/EuroSP.2018.00031
68. Pornin, T., Stern, J.P.: Digital signatures do not guarantee exclusive ownership. In: Ioannidis, J., Keromytis, A., Yung, M. (eds.) ACNS 2005: 3rd International Conference on Applied Cryptography and Network Security. Lecture Notes in Computer Science, vol. 3531, pp. 138–150. Springer Berlin Heidelberg, Germany, New York, NY, USA (Jun 7–10, 2005). https://doi.org/10.1007/11496137_10
69. Ristenpart, T., Yilek, S.: When good randomness goes bad: Virtual machine reset vulnerabilities and hedging deployed cryptography. In: ISOC Network and Distributed System Security Symposium – NDSS 2010. The Internet Society, San Diego, CA, USA (Feb 28 – Mar 3, 2010)
70. Romailler, Y., Pelissier, S.: Practical fault attack against the ed25519 and eddsa signature schemes. In: 2017 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC). pp. 17–24. IEEE (2017)
71. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Advances in Cryptology—CRYPTO'89 Proceedings 9. pp. 239–252. Springer (1990)
72. Shoup, V.: On the security of a practical identification scheme. Journal of Cryptology **12**(4), 247–260 (Sep 1999). https://doi.org/10.1007/s001459900056
73. of Standards, N.I., Technology: Digital signature standard (DSS). Tech. Rep. NIST FIPS 186-5, National Institute of Standards and Technology (2023)
74. Stern, J., Pointcheval, D., Malone-Lee, J., Smart, N.P.: Flaws in applying proof methodologies to signature schemes. In: Yung, M. (ed.) Advances in Cryptology – CRYPTO 2002. Lecture Notes in Computer Science, vol. 2442, pp. 93–110. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 18–22, 2002). https://doi.org/10.1007/3-540-45708-9_7
75. Tasso, É., De Feo, L., El Mrabet, N., Pontié, S.: Resistance of isogeny-based cryptographic implementations to a fault attack. In: International Workshop on Constructive Side-Channel Analysis and Secure Design. pp. 255–276. Springer (2021)
76. Weng, J., Liu, S., Chen, K., Li, X.: Identity-based key-insulated signature with secure key-updates. In: Information Security and Cryptology: Second SKLOIS Conference, Inscrypt 2006, Beijing, China, November 29-December 1, 2006. Proceedings 2. pp. 13–26. Springer (2006)
77. Yang, G., Duan, S., Wong, D.S., Tan, C.H., Wang, H.: Authenticated key exchange under bad randomness. In: Danezis, G. (ed.) FC 2011: 15th International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 7035, pp. 113–126. Springer Berlin Heidelberg, Germany, Gros Islet, St. Lucia (Feb 28 – Mar 4, 2012). https://doi.org/10.1007/978-3-642-27576-0_10
78. Yilek, S., Rescorla, E., Shacham, H., Enright, B., Savage, S.: When private keys are public: Results from the 2008 debian openssl vulnerability. In: Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement. pp. 15–27 (2009)
79. Zhou, Y., Cao, Z., Chai, Z.: Identity based key insulated signature. In: Information Security Practice and Experience: Second International Conference, ISPEC 2006, Hangzhou, China, April 11-14, 2006. Proceedings 2. pp. 226–234. Springer (2006)

Supplementary Material

A Adversary Models and Security Goals

A.1 Adversary Models

This section presents the classic *adversary models* am, which characterize an adversary's ability to learn and create message signature pairs.

Definition 5 (Classic Adversary Models [42]). *For a signature scheme with public key* pk *we define the adversary models* $\mathsf{am} \in \{\mathrm{KOA}, \mathrm{KMA}, \mathrm{gCMA}, \mathrm{dCMA}, \mathrm{aCMA}\}$ *against an arbitrary* PPT *adversary* $\mathcal{A}$ *as follows:*

Key Only Attack KOA*: Adversary* $\mathcal{A}$ *has access to the public key* pk*.*

Known Message Attack KMA*: Adversary* $\mathcal{A}$ *obtains* q *message signature pairs* (m_i, σ_i)*, with (uniformly) randomly sampled* $m_i \leftarrow\$ \mathcal{D}_M^{\mathsf{s}}$ *for* $1 \leq i \leq q$*.*

Chosen Message Attack CMA*: Adversary* $\mathcal{A}$ *obtains* q *message signature pairs* (m_i, σ_i)*. For the choice of messages, we have three categories:*

- *Generic Chosen Message Attack* gCMA*: Adversary* $\mathcal{A}$ *chooses a set of messages* before *accessing* pk *and before accessing the signatures.*
- *Direct Chosen Message Attack* dCMA*: Adversary* $\mathcal{A}$ *chooses a set of messages* after *accessing* pk *and before accessing the signatures.*
- *Adaptive Chosen Message Attack* aCMA*:* $\mathcal{A}$ *can repeatedly access a signature oracle for adaptively chosen messages. In particular,* $\mathcal{A}$ *knows all queued message signature pairs and* pk *during this process.*

We use the notation $m \leftarrow\$ \mathcal{D}_M^{\mathsf{s}}$, which emphasizes the sampling of a message $m \in \mathcal{D}_M$ according to some probability distribution (e.g., uniform random distribution) over message space $\mathcal{D}_M$.

A.2 Security Goals

First, we present the classical security goals sg considering unforgeability introduced by Goldwasser, Micali, and Rivest in 1984 [41]. Subsequently, we present security goals beyond unforgeability, systematically analysed in [47,17,21]. We present the corresponding game-based formalizations as part of our signature security framework in Section 4.1.

Unforgeability. Classically, we want a signature scheme where an adversary cannot create "new" valid message signature pairs. We consider security goals from the defender's perspective, in other words, in terms of unforgeability [42]. Note that we overload the $\in$ symbol, s.t. $m \in \mathcal{Q}$ also means that $\mathcal{Q}$ contains at least one tuple with message m (which simplifies the notation $\exists \sigma : (m, \sigma) \in \mathcal{Q}$).

Definition 6 (Unforgeability). *For a signature scheme* Σ *with key pair* $(\mathsf{sk}, \mathsf{pk})$ *let* $\mathcal{Q}$ *denote the set of message signature pairs that the adversary observes during the security experiment. We define the security goals* $\mathsf{sg} \in \{\mathrm{sEUF}, \mathrm{wEUF}, \mathrm{sSUF}, \mathrm{wSUF}, \mathrm{UUF}, \mathrm{UB}\}$ *considering* exclusive ownership *against an arbitrary* PPT *adversary* $\mathcal{A}$ *as follows:*

Strong Existential Unforgeability sEUF*: Adversary $\mathcal{A}$ cannot compute a valid message signature pair (m^*, σ^*) with $(m^*, \sigma^*) \notin \mathcal{Q}$.*

Weak Existential Unforgeability wEUF*: Adversary $\mathcal{A}$ cannot compute a valid message signature pair (m^*, σ^*) assuming there exists no σ such that $m^* \notin \mathcal{Q}$.*

Strong Selective Unforgeability sSUF*: Adversary $\mathcal{A}$ cannot compute a valid signature σ^* for a message $m^* \notin \mathcal{Q}$ a priori chosen by $\mathcal{A}$* after *observing* pk*.*

Weak Selective Unforgeability wSUF*: Adversary $\mathcal{A}$ cannot compute a valid signature σ^* for a message $m^* \notin \mathcal{Q}$ a priori chosen by $\mathcal{A}$* before *observing* pk*.*

Universal Unforgeability UUF*: Adversary $\mathcal{A}$ cannot compute a valid signature for any a priori sampled message $m^* \leftarrow\$ \mathcal{D}_M^{\mathsf{s}}$ with $m^* \notin \mathcal{Q}$.*

Total Break UB*: $\mathcal{A}$ is not able to compute the secret key sk of Σ.*

Regarding terminology, note that *non-malleability* and sEUF have the same definition [4,30]. Due to the insignificance of UUF in modern literature, we do not separate a weaker variant, in which the adversary receives m^* after interacting with its oracles. We present the corresponding security definition in Section 4.1.

Exclusive Ownership. The basic idea for all exclusive ownership variants is that a signature should not be valid under a different public key. The weak and strong variants follow the intuition that for the weak goals, the adversary has to output a secret key, whereas this is not the case for the strong variants. In Section 4 we detail the term "match" which implicitly checks that two keys $(\mathsf{sk}, \mathsf{pk})$ correspond to each other by introducing and describing $\mathsf{KCheck}()$.

Definition 7 (Exclusive Ownership). *For a signature scheme with public key* pk *let $\mathcal{Q}$ denote the set of message signature pairs that the adversary observes during the security experiment. We define the security goals* $\mathsf{sg} \in \{\text{msUEO}, \text{sUEO}, \text{wUEO}, \text{sCEO}, \text{wCEO}, \text{sDEO}, \text{wDEO}\}$ *considering* exclusive ownership *against an arbitrary* PPT *adversary $\mathcal{A}$ as follows:*

Malicious Strong Universal Exclusive Ownership msUEO*: Adversary $\mathcal{A}$ cannot compute messages m_1^*, m_2^*, signature σ^*, and public keys $\mathsf{pk}_1^* \neq \mathsf{pk}_2^*$, such that (m_1^*, σ^*) is valid under pk_1^* and (m_2^*, σ^*) is valid under pk_2^*.*

Strong Universal Exclusive Ownership sUEO*: Adversary $\mathcal{A}$ cannot compute a tuple $(\mathsf{pk}^*, m^*, \sigma^*)$, such that (m^*, σ^*) is valid under pk^*, where $\mathsf{pk} \neq \mathsf{pk}^*$ and $\sigma^* \in \mathcal{Q}$.*

Weak Universal Exclusive Ownership wUEO*: Adversary $\mathcal{A}$ cannot compute a tuple $(\mathsf{sk}^*, \mathsf{pk}^*, m^*, \sigma^*)$, such that sk^* and pk^* match and (m^*, σ^*) is valid under pk^*, where $\mathsf{pk} \neq \mathsf{pk}^*$ and $\sigma^* \in \mathcal{Q}$.*

Strong Conservative Exclusive Ownership sCEO*: Adversary $\mathcal{A}$ cannot compute a tuple $(\mathsf{pk}^*, m^*, \sigma^*)$, such that (m^*, σ^*) is valid under pk^*, where $\mathsf{pk} \neq \mathsf{pk}^*$ and $(m^*, \sigma^*) \in \mathcal{Q}$.*

Weak Conservative Exclusive Ownership wCEO*: Adversary $\mathcal{A}$ cannot compute a tuple $(\mathsf{sk}^*, \mathsf{pk}^*, m^*, \sigma^*)$, such that sk^* and pk^* match and (m^*, σ^*) is valid under pk^*, where $\mathsf{pk} \neq \mathsf{pk}^*$ and $(m^*, \sigma^*) \in \mathcal{Q}$.*

Strong Destructive Exclusive Ownership sDEO*: Adversary $\mathcal{A}$ cannot compute a tuple $(\mathsf{pk}^*, m^*, \sigma^*)$, such that (m^*, σ^*) is valid under pk^*, where $\mathsf{pk} \neq \mathsf{pk}^*$, $\sigma^* \in \mathcal{Q}$, and $(m^*, \sigma^*) \notin \mathcal{Q}$.*

Weak Destructive Exclusive Ownership wDEO*: Adversary $\mathcal{A}$ cannot compute a tuple $(\mathsf{sk}^*, \mathsf{pk}^*, m^*, \sigma^*)$, such that sk^* and pk^* match and (m^*, σ^*) is valid under pk^*, where $\mathsf{pk} \neq \mathsf{pk}^*$, $\sigma^* \in \mathcal{Q}$, and $(m^*, \sigma^*) \notin \mathcal{Q}$.*

Message Bounding. The idea is that no efficient adversary can create a valid signature for two different messages under the same public key. Otherwise, an adversary could replace the signed message after signing, allowing them to claim the signature is for a different message.

Definition 8 (Message Bounding). *For a signature scheme Σ the security goal* $\mathsf{sg} = \mathrm{MB}$ *considering* message bounding *against an arbitrary* PPT *adversary $\mathcal{A}$ is defined as follows:*

Message Bounding MB*: $\mathcal{A}$ cannot compute messages $m_1^* \neq m_2^*$, signature σ^*, and public key pk^* (for signature scheme Σ), such that both (m_1^*, σ^*) and (m_2^*, σ^*) are valid under pk^*.*

Note, we do not separate "strong" MB, which combines MB and msUEO.

Non-Resignability. The idea may seem counterintuitive: no efficient adversary can sign a message m with its keys by knowing public parameters and the signature σ, but *without knowing the message m*. However, this may be possible, e.g., if σ includes auxiliary data as the hash of m. For further information, including an explanation for the distributor $\mathcal{D}$ details, we refer to the literature [21].

Definition 9 (Non-Resignability). *For a signature scheme with key pair $(\mathsf{sk}, \mathsf{pk})$ and a* PPT *sampler $\mathcal{D}$ with computationally independent auxiliary data aux for KGen and conditional (HILL) min-entropy of output (m, aux) strictly greater than logarithmic in the security parameter λ, the security goal* $\mathsf{sg} = \mathrm{NR}$ *considering* non-resignability *against an arbitrary* PPT *adversary $\mathcal{A}$ is defined as follows:*

Non-Resignability NR*: Adversary $\mathcal{A}$ cannot compute a signature σ^* and a public key pk^* such that $\mathsf{pk}^* \neq \mathsf{pk}$ and (m, σ^*) verifies under pk^*, where (m, aux) is sampled by $\mathcal{D}(\lambda, \mathsf{pk})$, σ is a signature for m, and the adversary is given $(\mathsf{pk}, \sigma, \mathsf{aux})$.*

We specify the computational independence of aux from $\mathcal{D}$ as follows: A PPT distinguisher $\mathcal{A}$ cannot distinguish two random variables D_1 and D_2 from $\mathcal{D}$ with more than negligible advantage from $(\mathsf{sk}, \mathsf{pk}, m_0, \mathsf{aux}_b)$ where $D_b(\lambda)$, with $b \leftarrow\$ \{0, 1\}$, creates $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(\lambda)$, samples $(m_0, \mathsf{aux}_0) \leftarrow\$ \mathcal{D}(\lambda, \mathsf{pk})$ and $(m_1, \mathsf{aux}_1) \leftarrow\$ \mathcal{D}(\lambda, \mathsf{pk})$ [31].

BUFF Transformations. Cremers et al. [21] provide a generic security compiler to achieve malicious strong universal exclusive ownership msUEO, message bounding MB, and non-resignability NR from any wEUF-CMA-secure signature scheme $\Sigma = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ with keypair $(\mathsf{sk}, \mathsf{pk})$, which gives a new scheme $\Sigma' = (\mathsf{KGen}, \mathsf{Sign}', \mathsf{Vrfy}')$. The idea for Sign' is computing hash $h = \mathsf{H}(m, \mathsf{pk})$ with hash function H, signing $\sigma = \mathsf{Sign}(\mathsf{sk}, h)$ and outputting (σ, h). Then, on input $(\mathsf{pk}, m', (\sigma', h'))$, verification Vrfy' computes $h^* = \mathsf{H}(m, \mathsf{pk})$ and $b = \mathsf{Vrfy}(\mathsf{pk}, h', \sigma')$ and outputs 1, if $b = 1$ and $h^* = h'$ holds and 0 otherwise. Similarly to Cremers et al. [21], we provide generic security compilers in Section C and Section D, but in contrast, for adversary models beyond message choice, rather than security goals beyond unforgeability.

B Relations

B.1 Classical Signature Security Relations

We use the notation $\mathsf{sg}_s \rightarrow \mathsf{sg}_w$ to represent the relations, where $\mathsf{sg}_s \neq \mathsf{sg}_w$ and $\mathsf{sg}_w \not\rightarrow \mathsf{sg}_s$. Given a fixed adversary model (see Definition 5), the relation means that if sg_s is fulfilled, then sg_w is also implicitly met. We refer to sg_s as the stricter and sg_w as the less strict security goal. As a result, we present the relations of the security goals from Definition 6 as follows:

Lemma 1 (Relations Among Security Goals [42]). *Assume an arbitrary* PPT *adversary $\mathcal{A}$ with a fixed adversary model (see Definition 5). Then, the order of relation for the security goals from Section A.2 is as follows:*

$$\text{sEUF} \rightarrow \text{wEUF} \rightarrow \text{sSUF} \rightarrow \text{wSUF} \rightarrow \text{UUF} \rightarrow \text{UB}$$

Proof. Sequentially, we prove the relations of Lemma 1 for a PPT adversary $\mathcal{A}$ with a fixed adversary model as follows:

- UUF $\rightarrow$ UB*:* If $\mathcal{A}$ has access so sk and therefore is able to break UB, $\mathcal{A}$ can also break UUF, because $\mathcal{A}$ can use sk to compute arbitrary signatures.
- wSUF $\rightarrow$ UUF*:* If $\mathcal{A}$ can compute a new valid signature for an arbitrary message, $\mathcal{A}$ can do the same for a chosen message since this is a special case of random choice.
- sSUF $\rightarrow$ wSUF*:* If $\mathcal{A}$ can compute a new valid signature for a message chosen before acquiring pk, $\mathcal{A}$ can do the same after seeing pk since $\mathcal{A}$ implicitly does not need this knowledge.
- wEUF $\rightarrow$ sSUF*:* If $\mathcal{A}$ can compute a new valid signature for an a priori chosen message, $\mathcal{A}$ can do the same for a message chosen at any time since a priori is a special case of arbitrary choice.
- sEUF $\rightarrow$ wEUF*:* If $\mathcal{A}$ can compute a new valid signature for a chosen message that does not already have a signature, $\mathcal{A}$ implicitly can break sEUF because this also fulfills the condition for breaking sEUF.

wSUF $\not\rightarrow$ UUF, sSUF $\not\rightarrow$ wSUF, wEUF $\not\rightarrow$ sSUF, and sEUF $\not\rightarrow$ wEUF hold because $\rightarrow$ would be an implication of a general case to the same case with an additional stricter condition, which does not have to be fulfilled in the general case by default.

We show UUF $\not\rightarrow$ UB by counter-example. Assume a signature scheme under the hash and sign paradigm. Suppose an adversary $\mathcal{A}$ can obtain signatures for messages and can break the collision resistance of the used hash function. In that case, $\mathcal{A}$ can obtain σ for an arbitrary message m and then create m' with the same hash as m, leading to the forgery (m', σ). Since $\mathcal{A}$ does not require sk for this, only UUF and not UB is broken.

We adopt the previous notation $\mathsf{am}_s \rightarrow \mathsf{am}_w$, where $\mathsf{am}_s \neq \mathsf{am}_w$ and $\mathsf{am}_w \not\rightarrow \mathsf{am}_s$. However, since we are not setting security goals but adversary models in relation, the interpretation changes slightly. For a fixed security goal, the relation means that if $\mathcal{A}$ has the abilities of am_s, then $\mathcal{A}$ implicitly has the abilities of am_w. We refer to am_s as the stronger and am_w as the weaker adversary model. As a result, we present the relations of the adversary models as follows:

Lemma 2 (Relations Among Adversary Models [42]). *Let there be a probabilistic polynomial time* PPT *adversary $\mathcal{A}$. Then, the order of relation for the adversary models from Definition 5 is as follows:*

$$\text{aCMA} \rightarrow \text{dCMA} \rightarrow \text{gCMA} \rightarrow \text{KMA} \rightarrow \text{KOA}$$

Proof. We prove this for any PPT adversary $\mathcal{A}$ by using a simulation argument:

- KMA $\rightarrow$ KOA: KMA can simulate KOA because KOA is a special case of KMA, where the adversary does not queue the oracle.
- CMA $\rightarrow$ KMA: CMA can simulate KMA because KMA is a special case of CMA, where the adversary chooses the messages with the same probability distribution $\mathcal{D}_{M_{sample}}$ on which the experiment is defined.

Next, we prove the relations in which, for a given adversary model, the adversary's choice of parameters can be generic, direct or adaptive:

- Direct $\rightarrow$ generic: A fixed adversary with direct choice over a parameter can simulate generic choice by simply choosing a priori seeing the pk and sticking to that choice after seeing pk.
- Adaptive $\rightarrow$ direct: A fixed adversary with adaptive choice over a parameter can simulate direct choice by simply choosing a priori after seeing the pk and sticking to that choice while interacting with the oracle.

This implies that the relation chain aCMA $\rightarrow$ dCMA $\rightarrow$ gCMA holds because the parameter is the message to be chosen. Since the stronger adversary models can simulate the weaker ones, and the adversary models for a relation can not be equivalent, the reverse directions of the relations may not hold. That means KOA $\not\rightarrow$ KMA, KMA $\not\rightarrow$ gCMA, gCMA $\not\rightarrow$ dCMA, and dCMA $\not\rightarrow$ aCMA hold.

For the relations within the security definitions, $\mathsf{sd} = (\mathsf{sg}, \mathsf{am})$ we again use the notation $\mathsf{sd}_s \rightarrow \mathsf{sd}_w$, where $\mathsf{sd}_s \neq \mathsf{sd}_w$ and $\mathsf{sd}_s \not\rightarrow \mathsf{sd}_w$. We refer to sd_s as the stronger and sd_w as the weaker security definition. Figure 7 shows all possible relations for the security definitions. Lemma 3 states that these are all possible relations and that no further implications exist.

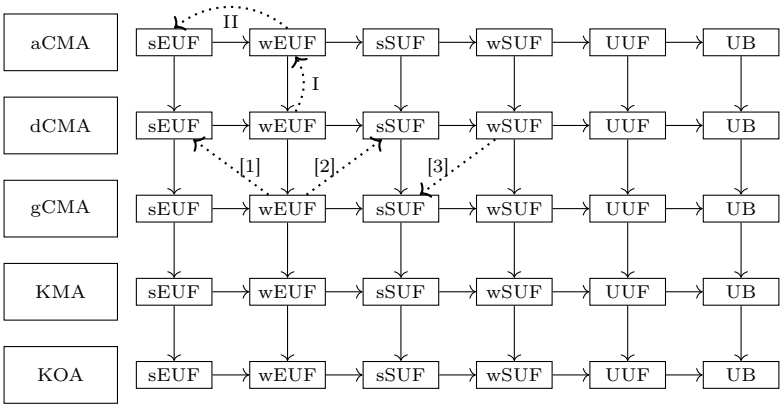


Fig. 7: Relations within the classical signature security model, where the rows represent the relation within sg and the columns those for am. The dotted implications are examples of relations that do not hold in general.

Lemma 3 (Restriction of Relations Within Security Definitions). *A relation between two security definitions* $\mathsf{sd}_s = (\mathsf{sg}_s, \mathsf{am}_s)$ *and* $\mathsf{sd}_w = (\mathsf{sg}_w, \mathsf{am}_w)$ *can exist if and only if* $\mathsf{sg}_s \rightarrow \mathsf{sg}_w$ *and* $\mathsf{am}_s \rightarrow \mathsf{am}_w$ *hold.*

Proof. Based on the results of Lemmas 1 and 2, we know that a weaker am can not imply a stronger am for a fixed sg (see example relation I in Figure 7). Similarly, a less strict sg can not imply a stricter sg for a fixed am (see example relation II in Figure 7). Since we know these implications are not possible, we reduce three cases (example relation [1], [2], and [3]):

- $(\mathsf{sg}_w, \mathsf{am}_w) \not\rightarrow (\mathsf{sg}_s, \mathsf{am}_s)$, where $\mathsf{sg}_s \rightarrow \mathsf{sg}_w$ and $\mathsf{am}_s \rightarrow \mathsf{am}_w$ (see example relation [1] in Figure 7): If $(\mathsf{sg}_w, \mathsf{am}_w) \rightarrow (\mathsf{sg}_s, \mathsf{am}_s)$ would hold, then $\mathsf{am}_w \rightarrow \mathsf{am}_s$ would also have to hold, which would conflict with relations like I. Moreover, $\mathsf{sg}_w \rightarrow \mathsf{sg}_s$ would have to hold, which would contradict relations like II. Thus $(\mathsf{sg}_w, \mathsf{am}_w) \rightarrow (\mathsf{sg}_s, \mathsf{am}_s)$ cannot hold.
- $(\mathsf{sg}_s, \mathsf{am}_w) \not\rightarrow (\mathsf{sg}_w, \mathsf{am}_s)$, where $\mathsf{sg}_s \rightarrow \mathsf{sg}_w$ and $\mathsf{am}_s \rightarrow \mathsf{am}_w$ (see example relation [2] in Figure 7): If $(\mathsf{sg}_s, \mathsf{am}_w) \rightarrow (\mathsf{sg}_w, \mathsf{am}_s)$ would hold, then $\mathsf{am}_w \rightarrow \mathsf{am}_s$ would also have to hold, which would contradict relations like I. Thus $(\mathsf{sg}_s, \mathsf{am}_w) \rightarrow (\mathsf{sg}_w, \mathsf{am}_s)$ cannot hold.
- $(\mathsf{sg}_w, \mathsf{am}_s) \not\rightarrow (\mathsf{sg}_s, \mathsf{am}_w)$, where $\mathsf{sg}_s \rightarrow \mathsf{sg}_w$ and $\mathsf{am}_s \rightarrow \mathsf{am}_w$ (see example relation [3] in Figure 7): If $(\mathsf{sg}_w, \mathsf{am}_s) \rightarrow (\mathsf{sg}_s, \mathsf{am}_w)$ would hold, then $\mathsf{sg}_w \rightarrow$

sg_s would also have to hold, which would contradict relations like II. Thus $(\mathsf{sg}_w, \mathsf{am}_w) \rightarrow (\mathsf{sg}_s, \mathsf{am}_s)$ cannot hold.

Relations Beyond Unforgeability. As pointed out by Cremers et al. [21], the security goals beyond unforgeability are mostly independent. Consequently, we were not able to identify any relations (beyond trivial ones) between weak and strong variants. We leave an analysis of the combination of multiple goals for future work.

B.2 General Relations Between Attack Vectors

In classical security frameworks such as digital signatures, adversary models (see Definition 5) are typically related by a partial order (see Lemma 2). This allows distinguishing *stronger* adversary models am_s from *weaker* models am_w. We write $\mathsf{am}_s \rightarrow \mathsf{am}_w$ to denote that am_s strictly implies am_w (i.e., $\mathsf{am}_s \neq \mathsf{am}_w$ and $\mathsf{am}_w \not\rightarrow \mathsf{am}_s$). That is, any adversary in model am_s has all the capabilities of an adversary in model am_w. Fixing a security goal sg, this implies: if a scheme is secure against am_s, then it is also secure against am_w. We lift this implication to sets of adversary models. Let $\overrightarrow{\mathsf{am}_S} = \{\mathsf{am}_1, \ldots, \mathsf{am}_n\}$ and $\overrightarrow{\mathsf{am}_W} = \{\mathsf{am}'_1, \ldots, \mathsf{am}'_m\}$ be two sets of adversary models. We write $\overrightarrow{\mathsf{am}_S} \rightarrow \overrightarrow{\mathsf{am}_W}$ if each model in $\overrightarrow{\mathsf{am}_W}$ is implied by some model in $\overrightarrow{\mathsf{am}_S}$.

Lemma 4. *Let $\overrightarrow{\mathsf{am}_S} = \{\mathsf{am}_1, \ldots, \mathsf{am}_n\}$ and $\overrightarrow{\mathsf{am}_W} = \{\mathsf{am}'_1, \ldots, \mathsf{am}'_m\}$ be sets of adversary models. Then $\overrightarrow{\mathsf{am}_S} \rightarrow \overrightarrow{\mathsf{am}_W}$ holds if and only if for every $\mathsf{am}'_j \in \overrightarrow{\mathsf{am}_W}$, there exists $\mathsf{am}_i \in \overrightarrow{\mathsf{am}_S}$ such that $\mathsf{am}_i = \mathsf{am}'_j$ or $\mathsf{am}_i \rightarrow \mathsf{am}'_j$.*

Proof (Sketch). Assume $\overrightarrow{\mathsf{am}_S} = \{\mathsf{am}_1, \ldots, \mathsf{am}_n\}$ and $\overrightarrow{\mathsf{am}_W} = \{\mathsf{am}'_1, \ldots, \mathsf{am}'_m\}$ are sets of adversary models such that for every $\mathsf{am}'_j \in \overrightarrow{\mathsf{am}_W}$ there exists $\mathsf{am}_i \in \overrightarrow{\mathsf{am}_S}$ with $\mathsf{am}_i = \mathsf{am}'_j$ or $\mathsf{am}_i \rightarrow \mathsf{am}'_j$. Let $\mathcal{A}$ be an adversary with access to the capabilities defined by $\overrightarrow{\mathsf{am}_S}$. For each am'_j in $\overrightarrow{\mathsf{am}_W}$, we construct a simulation of an adversary operating under am'_j using the capabilities of the corresponding $\mathsf{am}_i \in \overrightarrow{\mathsf{am}_S}$. Since $\mathsf{am}_i \rightarrow \mathsf{am}'_j$, any action or oracle available in am'_j can be emulated within am_i. As this holds for every $\mathsf{am}'_j \in \overrightarrow{\mathsf{am}_W}$, adversary $\mathcal{A}$ with access to $\overrightarrow{\mathsf{am}_S}$ can fully simulate any adversary with access to $\overrightarrow{\mathsf{am}_W}$. Thus, if a scheme is secure against all adversaries in $\overrightarrow{\mathsf{am}_S}$, it is secure against those in $\overrightarrow{\mathsf{am}_W}$.

In Section B.1, we present the implication relations for signature schemes. The relations for randomness-based attacks are analogous to those for message-based attacks shown in Figure 7. These relations are relatively straightforward. However, we were unable to identify more fine-grained or advanced relations without further assumptions about the structure of the scheme. This limitation applies to both the generalized adversarial model and the extended framework for signatures. Following the implication direction established by the Lemma 4, we conservatively consider the strongest instances of each attack vector when analyzing general constructions in the subsequent sections.

Exp-Secrecy(λ)	Exp-Weak-NRep(λ)	Exp-Strong-NRep(λ)
$1: \ \mathcal{Q} \leftarrow \emptyset;$	$1: \ \mathcal{Q} \leftarrow \emptyset;$	$1: \ \mathcal{Q} \leftarrow \emptyset;$
$2: \ (\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \ \mathsf{KGen}(\lambda)$	$2: \ (\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \ \mathsf{KGen}(\lambda)$	$2: \ (\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \ \mathsf{KGen}(\lambda)$
$3: \ \mathsf{tr}^* \leftarrow\$ \ \mathcal{A}^{\mathcal{O}^{\mathsf{am}_i}_{\mathsf{A}_i}(\mathsf{sk},\cdot;\cdot)}(\mathsf{pk})$	$3: \ (j,k) \leftarrow\$ \ \mathcal{A}^{\mathcal{O}^{\mathsf{am}_i}_{\mathsf{A}_i}(\mathsf{sk},\cdot;\cdot)}(\mathsf{pk})$	$3: \ (j,k) \leftarrow\$ \ \mathcal{A}^{\mathcal{O}^{\mathsf{am}_i}_{\mathsf{A}_i}(\mathsf{sk},\cdot;\cdot)}(\mathsf{pk})$
$4: \ \textbf{return } \mathsf{tr}^* \in \mathcal{Q}^{\mathsf{tr}}_{\mathsf{A}_i}$	$4: \ (m_j, r_j, \mathsf{tr}_j) \leftarrow \mathcal{Q}[j]$	$4: \ (m_j, r_j, \mathsf{tr}_j) \leftarrow \mathcal{Q}[j]$
	$5: \ (m_k, r_k, \mathsf{tr}_k) \leftarrow \mathcal{Q}[k]$	$5: \ (m_k, r_k, \mathsf{tr}_k) \leftarrow \mathcal{Q}[k]$
	$6: \ b_0 \leftarrow (\mathsf{tr}_j = \mathsf{tr}_k)$	$6: \ \textbf{return } \mathsf{tr}_j = \mathsf{tr}_k$
	$7: \ b_1 \leftarrow ((m_j, r_j) \neq (m_k, r_k))$	
	$8: \ \textbf{return } b_0 \wedge b_1$	

Fig. 8: Security experiment for secrecy of the internal randomness.

C Mitigating Bad Randomness Attacks

In this section, we formally define three general security goals for the randomness of an algorithm. These are *secrecy*, *weak non-repeatability*, and *strong non-repeatability*. Then we build a compiler that allows us to achieve all three goals for any cryptographic primitive matching the type annotation of our model. Thus, this section formalizes, streamlines, and generalizes the results of Alekseev et al. [2] considering message authentication with untrusted randomness, Bellare and Tackmann [13] analyzing nonce-based cryptography, Fischlin et al. [37], who instantiate GenTr with hash function similar to $\mathsf{PRF}(\mathsf{sk}, \mathit{input}; r)$ and Ristenpart and Yilek [69], who instantiate GenTr with a hedging computation similar to $\mathsf{PRF}(\mathsf{sk}, \mathit{input}; r)$, into consistent definitions.

Towards Security Goals for Randomness. According to our model, we divide the randomness generator into two parts: the untrusted process GenR, which generates a random element r, and the trusted subroutine GenTr, which conveys the randomness r into temporary randomness tr. Our model assumes that an adversary $\mathcal{A}$ controls the randomness r. When we consider security goals for randomness, we define properties for the temporary value tr which will be used for the deterministic computation, when $\mathcal{A}$ has full control over r. Then, during interaction between $\mathcal{A}$ and some algorithm A_i according to some adversary model, $\mathcal{A}$ should not be able to break these properties for tr. Regarding the specification of these security goals for randomness, we consider two dimensions that are orthogonal to each other, namely *secrecy* and *non-repeatability*.

Secrecy. On a high level, secrecy guarantees that no efficient adversary can learn the "internal randomness" tr after interacting with an algorithm A_i. We provide the corresponding experiment $\mathsf{Exp\text{-}Secrecy}$ in Figure 8.

Non-Repeatability. On a high level, non-repeatability guarantees that no efficient adversary can enforce an algorithm to use the same "working randomness" tr as input to the deterministic computation. For non-repeatability, we distinguish

two cases: First, we consider *weak non-repeatability*, which guarantees that each tr differs, if the input to GenTr differs. In addition, we consider *strong non-repeatability* which guarantees that tr differs on each invocation of an algorithm, even if the inputs to GenTr are consistent. In Figure 8, we define both experiments, $\mathsf{Exp\text{-}Weak\text{-}NRep}$ for weak non-repeatability and $\mathsf{Exp\text{-}Strong\text{-}NRep}$ for strong non-repeatability. We summarize the relations between these notions in the following lemma.

Lemma 5. *Strong non-repeatability implies weak non-repeatability.*

Proof. The implication from strong to weak non-repeatability is immediate from the experiments in Figure 8. For any adversary $\mathcal{A}$, the winning event in $\mathsf{Exp\text{-}Weak\text{-}NRep}$ is

$$(\mathsf{tr}_j = \mathsf{tr}_k) \wedge ((m_j, r_j) \neq (m_k, r_k)),$$

which is a subset of the winning event in $\mathsf{Exp\text{-}Strong\text{-}NRep}$, namely

$$\mathsf{tr}_j = \mathsf{tr}_k.$$

Hence, if the probability of winning $\mathsf{Exp\text{-}Strong\text{-}NRep}$ is negligible for all PPT adversaries, then so is the probability of winning $\mathsf{Exp\text{-}Weak\text{-}NRep}$. □

In contrast, secrecy is independent of non-repeatability. A scheme may hide tr perfectly while still reusing it across invocations, and conversely, it may guarantee fresh values of tr on every invocation while revealing them to the adversary. Therefore, secrecy neither implies nor is implied by weak or strong non-repeatability. Since secrecy inherently depends on algorithm-specific behavior that is abstracted away in our generalized analysis, we do not consider it further.

Towards A Compiler. Having defined security for cryptographic schemes under bad randomness, our goal is to construct a compiler that transforms existing schemes to meet the requirements of secrecy and non-repeatability. Some cryptographic schemes, such as CL signatures [19], inherently lack secrecy and fail to ensure non-repeatability when an adversary controls the randomness. To address these issues, our compiler uses pseudorandom functions PRFs as a foundational building block and instantiates the GenTr algorithm. The compiler preserves the correctness of the original scheme since the internal randomness used after our compiler and the initial internal randomness live in the same space.

A first natural idea is to derive the temporary randomness only from the adversarial randomness, i.e., to instantiate GenTr as $\mathsf{tr} \leftarrow \mathsf{PRF}(\mathsf{sk}; r)$. However, this is not sufficient for a generic compiler. For Schnorr signatures (see Section E), if an adversary queries two different messages $m_1 \neq m_2$ with the same randomness input r, then both signatures use the same nonce tr. Writing the signatures as $s_1 = \mathsf{tr} + H(m_1, R)\mathsf{sk}$ and $s_2 = \mathsf{tr} + H(m_2, R)\mathsf{sk}$, the adversary obtains $s_1 - s_2 = \big(H(m_1, R) - H(m_2, R)\big)\mathsf{sk}$, and can therefore recover sk. Thus, a compiler that only depends on r does not generically preserve security. Conversely, deriving the temporary randomness only from the input, e.g., $\mathsf{tr} \leftarrow \mathsf{PRF}(\mathsf{sk}; m)$, is also not sufficient. Again, in the case of Schnorr signatures, two invocations

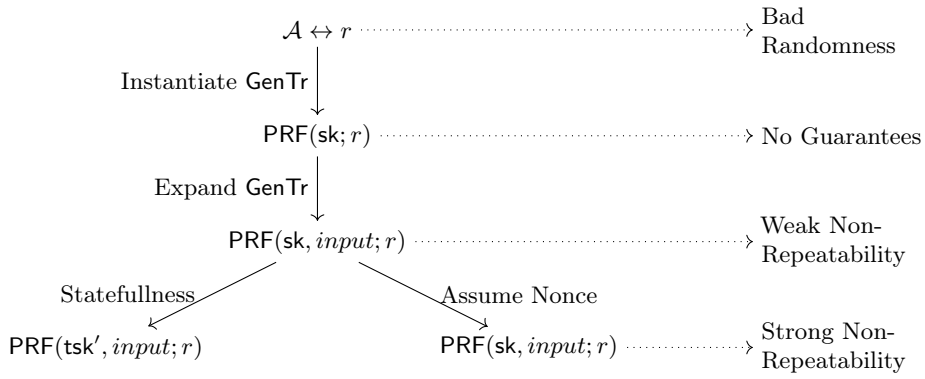


Fig. 9: Visual guide for using PRF against bad randomness.

on the same message m always yield the same tr and the adversary can proceed as above. Therefore, neither input m nor randomness r alone is sufficient for a generic compiler. As a result, our generic construction must depend on both and derive tr from the pair (m, r).

This directly gives us *weak non-repeatability* by computing $\mathsf{tr} \leftarrow \mathsf{PRF}(\mathsf{sk}, input; r)$. This approach re-randomizes tr with each execution, ensuring non-repeatability. However, in case the adversary has adaptive control over the input and the randomness, the adversary can fix both over multiple queries, resulting in the same internal randomness. Consequently, for *strong non-repeatability*, we deviate into two different solutions: for the first solution, we assume that r is a nonce. This means that the adversary controls the values of r but has the limitation that each value of r can only be submitted once. This assumption is in accordance with the threat model in which an adversary compromises a device's random number generator, yet remains constrained by its functionality [20,23,46]. By leveraging the assumption that r is a nonce input to the PRF, we ensure strong non-repeatability of our cryptographic scheme. A different approach without a nonce would be a stateful computation, where the inputs of the previous execution are used to update a temporary signing key tsk after each computation, which is similar to the approach of Bellare and Tackmann [13]. We now formally claim the security of our compiler in Theorem 3 and present the security goals it achieves in Figure 9. Since the stronger mc and rc imply the weaker ones (see Section B.2), our generalized theorem uses the strongest instantiations of mc and rc.

Theorem 3 (PRF **Secures Against Bad Randomness).** *Let $\Pi = (\mathsf{A}_0, \ldots, \mathsf{A}_\lambda)$ be a cryptographic scheme w.r.t. Model 3.1 satisfying a security goal* sg *against any* PPT *adversary $\mathcal{A}$ with adaptive chosen input* aCMA*. Let us further assume that Π satisfies this security goal, while* GenTr *computes the identity function (i.e.,* $\mathsf{tr} = r$*),* sk *is secret, and r fulfills secrecy, weak (strong) non-repeatability (i.e., r is not trivially leaked by the scheme). Let $F\colon \{0,1\}^\lambda \times \{0,1\}^* \to \mathcal{D}_R$ be a*

$\mathcal{O}^{\mathrm{aCMAaCRA}}_{1,\mathsf{A}_i}(\mathsf{sk}, m; r)$	$\mathcal{O}^{\mathrm{aCMAaCRA}}_{2,\mathsf{A}_i}(\mathsf{sk}, m; r)$	$\mathcal{O}^{\mathrm{aCMAaCRA}}_{3,\mathsf{A}_i}(\mathsf{sk}, m; r)$
1 : parse sk as $(\mathsf{sk}_1, \mathsf{sk}_2)$	1 : parse sk as $(\mathsf{sk}_1, \mathsf{sk}_2)$	1 : parse sk as $(\mathsf{sk}_1, \mathsf{sk}_2)$
2 : $\mathsf{tsk} \leftarrow F(\mathsf{sk}_1, m; r)$	2 : **if** $\mathcal{Q}_2[(\mathsf{sk}, m, r)] = \bot$	2 : **if** $\mathcal{Q}_3[(\mathsf{sk}, m, r)] = \bot$
3 : $\sigma \leftarrow \mathcal{O}_{\mathsf{A}_i}(\mathsf{sk}_2, m; \mathsf{tsk})$	3 : $\mathsf{tsk} \leftarrow\$ \mathcal{D}_R$	3 : $\sigma \leftarrow \mathcal{O}^{\mathrm{aCMA}}_{\mathsf{A}_i}(\mathsf{sk}_2, m)$
4 : **return** σ	4 : $\mathcal{Q}_2[(\mathsf{sk}, m, r)] \leftarrow \mathsf{tsk}$	4 : $\mathcal{Q}_3[(\mathsf{sk}, m, r)] \leftarrow \sigma$
	5 : **else**	5 : **else**
	6 : $\mathsf{tsk} \leftarrow \mathcal{Q}_2[(\mathsf{sk}, m, r)]$	6 : $\sigma \leftarrow \mathcal{Q}_3[(\mathsf{sk}, m, r)]$
	7 : **return** $\mathcal{O}_{\mathsf{A}_i}(\mathsf{sk}_2, m; \mathsf{tsk})$	7 : **return** σ

Fig. 10: Oracle transition from $\mathcal{O}^{\mathrm{aCMAaCRA}}_{\mathsf{A}_i}$ to $\mathcal{O}^{\mathrm{aCMA}}_{\mathsf{A}_i}$. The highlighted rows are only part of the oracle in reducing secrecy and weak non-repeatability but not for strong non-repeatability. For secrecy, we additionally ignore the message when checking the queues in Line 2, and do not input the message to the PRF. We omit filling the queues of the internal state for better readability.

PRF. *Then, Π satisfies* sg *against any* PPT *adversary $\mathcal{A}$ with* aCMA *and also with adaptive chosen randomness* aCRA*, if we modify* GenTr *into*

- $\mathsf{tr} \leftarrow F(\mathsf{sk}, m; r)$ *for weak non-repeatability, and either*
- $\mathsf{tr} \leftarrow F(\mathsf{sk}, m; r)$ *with the nonce assumption for r, or*
- $\mathsf{tr} \leftarrow F(\mathsf{tsk}', m; r)$ *assuming derandomisation for strong non-repeatability.*

Proof. We prove our theorem by a series of game hops providing an oracle transition (cf. Figure 10) from adaptive chosen message and adaptive chosen randomness (aCMAaCRA) to adaptive chosen message (aCMA). In both cases, the proof follows the same general pattern: we first replace the PRF in the compiled oracle by a truly random function and bound the resulting gap by the pseudorandomness of F, and then show that the resulting oracle is equivalent to an aCMA oracle. We assume throughout that an aCMA oracle samples a fresh computational randomness on each invocation and that the collision probability of two uniform samples from $\mathcal{D}_R$ is negligible.

For *weak non-repeatability*, we consider the compiler $\mathsf{tr} \leftarrow F(\mathsf{sk}_1, m; r)$. The relevant winning condition is that two invocations with distinct pairs $(m_j, r_j) \neq (m_k, r_k)$ lead to the same computational randomness. Accordingly, we model the oracle transition such that the aCMA oracle is invoked once for every fresh message-randomness pair (m, r), while repeated queries to the same pair are answered consistently from a queue. In Figure 10, the first oracle $\mathcal{O}^{\mathrm{aCMAaCRA}}_{1,\mathsf{A}_i}$ is exactly the oracle expected by the adversary in the compiled (aCMAaCRA) game, and the second oracle $\mathcal{O}^{\mathrm{aCMAaCRA}}_{2,\mathsf{A}_i}$ replaces the PRF by a truly random function on input (m, r), again with consistency for repeated pairs. Assume towards contradiction that there is a non-negligible gap between these two oracles. Then we build a distinguisher $\mathcal{B}$ against the pseudorandomness of F that simulates the security game for $\mathcal{A}$ using its own oracle $\mathcal{O}_{\mathcal{B}}$, which is either F or a truly random function. On input (m, r), $\mathcal{B}$ checks whether the pair was queried before. If so, it

answers consistently from the queue. Otherwise, it invokes $\mathcal{O}_{\mathcal{B}}$ on (m, r), uses the output as computational randomness tr, and forwards tr to the underlying algorithm. If $\mathcal{O}_{\mathcal{B}}$ is instantiated by F, then $\mathcal{B}$ simulates $\mathcal{O}^{\mathrm{aCMAaCRA}}_{1,\mathsf{A}_i}$ perfectly. If $\mathcal{O}_{\mathcal{B}}$ is a truly random function, then $\mathcal{B}$ simulates $\mathcal{O}^{\mathrm{aCMAaCRA}}_{2,\mathsf{A}_i}$ perfectly. Thus, any non-negligible gap between the two games yields a non-negligible distinguisher against the PRF, contradicting the assumption on F. The transition from $\mathcal{O}^{\mathrm{aCMAaCRA}}_{2,\mathsf{A}_i}$ to $\mathcal{O}^{\mathrm{aCMAaCRA}}_{3,\mathsf{A}_i}$ is then only a rewriting: the aCMA oracle samples fresh randomness on every fresh invocation, and the same consistency checks are kept for repeated pairs. Hence, there is no gap between these two oracles. It follows that the probability of obtaining the same computational randomness on two distinct pairs (m, r) is bounded by the collision probability of two uniform values from $\mathcal{D}_R$, which is negligible.

For *strong non-repeatability*, we distinguish the two compiler instantiations from the theorem. In the nonce case, we again use $\mathsf{tr} \leftarrow F(\mathsf{sk}_1, m; r)$ and assume that the adversary may choose r adaptively, but may never repeat a value of r. In the derandomization case, we use $\mathsf{tr} \leftarrow F(\mathsf{tsk}', m; r)$ and assume that the temporary key tsk' is updated after every invocation, so that the PRF input changes even if (m, r) is repeated. In both cases, the relevant winning condition is that two distinct invocations lead to the same computational randomness. Accordingly, we model the oracle transition such that the aCMA oracle is invoked freshly on every invocation. In Figure 10, the first oracle is again the compiled (aCMAaCRA) oracle, while the second oracle replaces the PRF by a truly random function on the full fresh input of the invocation: in the nonce case freshness is provided by the non-repeating value r, and in the derandomization case by the updated state represented abstractly by the queue. Assume towards contradiction that there is a non-negligible gap between these two oracles. We again build a distinguisher $\mathcal{B}$ against the pseudorandomness of F. On every invocation, $\mathcal{B}$ forwards the corresponding fresh PRF input to its oracle $\mathcal{O}_{\mathcal{B}}$ and uses the answer as computational randomness tr. If $\mathcal{O}_{\mathcal{B}}$ computes F, then $\mathcal{B}$ simulates the first oracle perfectly. If $\mathcal{O}_{\mathcal{B}}$ is a truly random function, then $\mathcal{B}$ simulates the second oracle perfectly. Thus, any non-negligible gap again contradicts the pseudorandomness of F. The transition from the second oracle to the third oracle is, as before, only a rewriting: by assumption, the aCMA oracle samples fresh randomness on every invocation, and our modeling invokes it freshly on every invocation as well. Hence, there is no gap between these two oracles. It follows that the probability of obtaining the same computational randomness in two distinct invocations is bounded by the collision probability of two uniform values from $\mathcal{D}_R$, which is negligible. □

D Securing Against Key Exposure

Our goal is to prevent an adversary from generating verifiable output for new arbitrary messages after exposing tsk during the computation of $\mathsf{DetComp}$. This is challenging because, once a key is leaked, an attacker can use it to produce valid outputs without restriction, making traditional security goals (Section A.2)

$\mathcal{O}_f^{L_{\mathsf{tsk}}^{m,\sigma}}(m,\sigma)$

1 : $t = \{(m', \sigma', \cdot, \mathsf{tsk}') \in \mathcal{Q} \mid m' = m \wedge \sigma' = \sigma\}$, $\mathsf{tsk}' \leftarrow t$

2 : **if** $(m^* \neq m)$: **return** tsk', **else** : **return** $\perp$

Fig. 11: Leakage oracle for key exposure, which on input m and σ outputs the corresponding temporary secret key tsk. We assume that queue $\mathcal{Q}$ stores the values of every computation and that a global challenge message m^* may exist.

difficult to maintain. A common model considering a form of full key leakage is forward security, which ensures that computations performed before key leakage remain secure. This does not protect outputs generated after a compromise, nor does it provide a mechanism to detect key leakage. In contrast, key insulation approaches [28,27] aim to provide security even after a key exposure event. Existing solutions typically assume time-based key updates, meaning that secret keys evolve across predefined time periods. However, during a compromised period, there is no possibility to differentiate between signatures produced by a legitimate user and those generated by an adversary. Our goal is to also prohibit an adversary from creating any verifiable output for a new arbitrary input after key leakage. To achieve this, we focus on public key schemes and assume secure randomness. Finding a solution for our goal involves addressing four key challenges:

- Hiding: tsk does not reveal any information about the secret key sk.
- Binding to sk: tsk should be verifiably tied to sk, ensuring that an adversary cannot forge arbitrary values for tsk.
- Binding to m: tsk must be constrained to a specific input m (or set of inputs), preventing an adversary from misusing tsk with other values.
- Public verifiability: tsk (or similar public values) is verifiable under a (temporary) public verification key vk.

These requirements define a new generalized security goal, similar to weak existential unforgeability wEUF, which we call *unforgeable key exposure* UKE. We assume that any public key scheme with an algorithm A_i, to which an adversary has oracle access, includes a verification algorithm A_j. This verification ensures that an input-output pair (m, σ) is valid for A_i, i.e., $\mathsf{A}_j(\mathsf{pk}, \mathsf{A}_i(\mathsf{sk}, m; r)) = 1$ for a key pair $(\mathsf{sk}, \mathsf{pk})$, message $m \in \mathcal{D}_M$, and randomness $r \in \mathcal{D}_R$. We define UKE over multiple temporary secret keys and permit arbitrary secret-key leakage. A PPT adversary wins only if it outputs a valid pair for a new input (with respect to the underlying goal) and the key used for the final output was not leaked to it. The latter restriction is necessary to preserve generality and keep UKE meaningful across stronger goals such as strong existential unforgeability (sEUF): after learning a key, an adversary can locally recompute a fresh, independent output for the same input (e.g., re-randomize a signature) and otherwise obtain a trivial win. Hence, we assume a leakage oracle $\mathcal{O}_{\mathsf{Sign}}^{L_{\mathsf{tsk}}^{m,\sigma}}(m, \sigma)$, which on input m

$\mathsf{A}'_1(\mathsf{sk}, m)$	$\mathsf{A}'_2(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma')$	$\mathsf{GenTsk}(\mathsf{sk}, m)$
$r \leftarrow\$ \mathcal{D}_R$	$b' \leftarrow \mathsf{Vrfy}(\mathsf{pk}, m\|\|\mathsf{vk}, \sigma')$	$r \leftarrow\$ \mathcal{D}_\lambda$
$(\mathsf{tsk}, \mathsf{vk}, \sigma') \leftarrow \mathsf{GenTsk}(\mathsf{sk}, m)$	$b \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$	$(\mathsf{tsk}, \mathsf{vk}) \leftarrow \mathsf{A}_0(r)$
$\sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$	**return** $b \wedge b'$	$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m\|\|\mathsf{vk})$
return $(\mathsf{vk}, \sigma, \sigma')$		**return** $(\mathsf{tsk}, \mathsf{vk}, \sigma)$

Fig. 12: Overview of the construction of the key exposure security compiler.

and σ outputs the temporary secret key tsk but does not include challenge inputs m and/or outputs σ specific to some security goals. We present the resulting leakage oracle formalization in Figure 11.

A Message Compiler. Dodis et al. [28] proposed a generic $(N\text{-}1, N)$-secure signature scheme, which for N time periods tolerates up to N-1 corruption while at least one period remains secure. Their signature scheme achieves key insulation by relying on a trusted signing environment for securing key updates across time periods. We observe that we can adapt and generalize their technique to our setting by replacing time periods with messages: the derivation of a message-specific key tsk must be authenticated using a separate mechanism that remains outside the adversary's control. This leads to the following insight: in the absence of state or key evolution, to provide security for an arbitrary scheme, we need to delegate trust to a secure signature computation. We require sEUF-aCMA security for this signature scheme. Following a similar intuition as for UKE (prohibit trivial win by re-randomizing a signature), we consider sEUF. Further, aCMA follows from being the strongest goal for message choice and to provide the reduction with the necessary capabilities in the security proof. We present the resulting compiler in Figure 12. Subsequently, we show that it is possible to reduce the number of trusted computations, improving upon the naive adaptation of Dodis et al.'s construction.

Theorem 4 (Compiler Against Key Exposure). *Let $\Pi = (\mathsf{A}_0, \mathsf{A}_1, \mathsf{A}_2)$ be a cryptographic scheme w.r.t. Model 3.1 satisfying a security goal* sg *against any* PPT *adversary $\mathcal{A}$ with adaptive chosen input attack* aCMA *against algorithm* A_1 *and corresponding public verification algorithm* A_2*. Let $\Sigma = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ be a* sEUF-aCMA *secure signature scheme. Let* GenTsk *be the security compiler w.r.t. our Model in Section 3.1. Let $\Pi' = (\mathsf{A}_0, \mathsf{A}'_1, \mathsf{A}'_2)$ be the scheme obtained from Π by applying the security compiler from Figure 12. Then, Π' satisfies* sg *with* UKE *against any* PPT *adversary $\mathcal{A}$ with* aCMA *and a key exposure oracle.*

Proof. We begin by providing a high-level overview of our game-hopping proof. For a formal overview of the outlined games, see Figure 13 for reference. Our initial game G_1 is the original sg-$(\text{aCMA}, \mathcal{O}_L)$ game, which includes our security compiler to secure an arbitrary scheme Π against key exposure. Our second game G_2 catches an edge case, in which there is a collision between the main public key

$\mathsf{G}_1(\lambda), \mathsf{G}_2(\lambda), \mathsf{G}_3(\lambda)$

$(\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \mathsf{Gen}(1^\lambda)$
$\mathsf{vk}^*,\ \mathcal{Q}_\Sigma,\ \mathcal{Q}_L \leftarrow \emptyset$
$(m_1^*, \sigma_1^*, \sigma_2^*) \leftarrow\$ \mathcal{A}^{\mathcal{O}_1(\mathsf{sk},\cdot,\cdot),\mathcal{O}_2(\mathsf{pk},\cdot,\cdot,\cdot,\cdot),\mathcal{O}_L(\cdot,\cdot)}(\lambda, \mathsf{pk})$
if $(m_1^*, \sigma_1^*, \mathsf{vk}^*) \notin \mathcal{Q}_\Sigma \wedge \mathsf{Vrfy}(\mathsf{pk}, m_1^* || \mathsf{vk}^*, \sigma_1^*)$: **abort** G_3
return $\mathsf{Check}_{\mathsf{sg}}^{\Pi}()$

$\mathcal{O}_1(\mathsf{sk}, m, b)$

$r \leftarrow\$ \mathcal{D}_R$
$(\mathsf{tsk}, \mathsf{vk}, \sigma') \leftarrow \mathsf{GenTsk}(\mathsf{sk}, m)$
$\sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$
if $b = \top \wedge \mathsf{vk}^* = \emptyset$:
  $\mathsf{vk}^* \leftarrow \mathsf{vk}$
else :
  $\mathcal{Q}_L \leftarrow \mathcal{Q}_L \cup \{m, \sigma, \mathsf{tsk}\}$
return $(\mathsf{vk}, \sigma, \sigma')$

$\mathsf{GenTsk}(\mathsf{sk}, m)$

$r \leftarrow\$ \mathcal{D}_\lambda$
$\mathsf{tsk}, \mathsf{vk} \leftarrow \mathsf{A}_0(r)$
if $\mathsf{pk} = \mathsf{vk}$: **abort** $\mathsf{G}_{>=2}$
$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m || \mathsf{vk})$
$\mathcal{Q}_\Sigma \leftarrow \mathcal{Q}_\Sigma \cup \{(m, \sigma, \mathsf{vk})\}$
return $(\mathsf{tsk}, \mathsf{vk}, \sigma)$

$\mathcal{O}_2(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma')$

$b' \leftarrow \mathsf{Vrfy}(\mathsf{pk}, m || \mathsf{vk}, \sigma')$
$b \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$
return $b \wedge b'$

$\mathcal{O}_L(m, \sigma)$

$t = \{(m', \sigma', \mathsf{tsk}') \in \mathcal{Q}_L$
  $\mid m' = m \wedge \sigma' = \sigma\}$
return t

Fig. 13: Overview of the algorithms in the game hopping proof for Theorem 4. $\mathsf{Check}_{\mathsf{sg}}^{\Pi}()$ is a generic function that has as input the full transcript of the security game and outputs 1 iff the actions of the adversary break the security goal sg.

pk and the ephemeral verification keys vk. We bound such a collision (and hence, a gap between the games) to be negligible in the security parameter. Our third and last game G_3 catches the case in which the adversary outputs a forgery on a tuple containing a message–ephemeral-key pair. In the hop from G_2 to G_3, we argue that this forgery is only possible if the adversary breaks the sEUF-aCMA security of Σ, which contradicts our assumption. Consequently, we can finish the proof by analysing the sg-(aCMA, $\mathcal{O}_L$) of G_3 under the assumption that the output message m^* and the challenge key vk are non-fresh. If Game G_3 does not abort (which it does not with overwhelming probability), we reduce the security of Π satisfying security goal sg under an adaptively chosen input aCMA and key exposure attack to the security of Π satisfying sg under aCMA attacks *without key exposure*.

Game G_1*:* Our first game G_1 is the original sg-(aCMA, $\mathcal{O}_L$) game. It holds that $\Pr[\mathsf{sg}\text{-}(\mathrm{aCMA}, \mathcal{O}_L)_{\Pi}^{\mathcal{A}}(\lambda) = 1] = \Pr[\mathsf{G}_1(\lambda) = 1]$. To simplify the notation of G_1, we introduce a generic function $\mathsf{Check}_{\mathsf{sg}}^{\Pi}()$ which has as input the full transcript

of the security game and outputs 1 iff the actions of the adversary break the security goal sg.

Game G_2*:* The second game G_2 only differs from G_1 if the ephemeral verification key vk equals the main public key pk (see Line 3 of $\mathsf{GenTsk}(\mathsf{sk}, m)$ in Figure 13). We call this event E_1. It holds that

$$\Pr[\mathsf{G}_2(\lambda) = 1] = \Pr[\mathsf{G}_2(\lambda) = 1 \wedge E_1] + \Pr[\mathsf{G}_2(\lambda) = 1 \wedge \neg E_1].$$

Claim 1. *Let E_1 be the event, in which G_2 aborts in Line 3 of $\mathsf{GenTsk}(\mathsf{sk}, m)$. Let $\mathcal{D}_{\Pi}^{\mathsf{pk}}$ be the public key space of Π and let $\mathcal{D}_{\Sigma}^{\mathsf{pk}}$ be the public key space of Σ. Let q be the number of queries from adversary $\mathcal{A}$ to oracle $\mathcal{O}_1$. Then, $\Pr[\mathsf{G}_2(\lambda) = 1 \wedge E_1] \leq q/|\mathcal{D}_{\Pi}^{\mathsf{pk}}|$, which is negligible.*

Proof (of Claim 1). We apply the union bound over all q queries to bound the total probability of the event E_1. It holds, that $\Pr[\mathsf{G}_2(\lambda) = 1 \wedge E_1] \leq \Pr[E_1]$. Assuming that pk is chosen uniformly at random from $\mathcal{D}_{\Pi}^{\mathsf{pk}}$, for the total probability of the event E_1, it holds that $\Pr[E_1] \leq \sum_{i=1}^{q} \Pr[\mathsf{pk} = \mathsf{vk}_i] = q/|\mathcal{D}_{\Pi}^{\mathsf{pk}}|$. Since the number of queries q is polynomial in the security parameter λ while the size of the key space $|\mathcal{D}_{\Pi}^{\mathsf{pk}}|$ is exponential, the value $q/|\mathcal{D}_{\Pi}^{\mathsf{pk}}|$ is negligible, and so is $\Pr[\mathsf{G}_2(\lambda) = 1 \wedge E_1]$. □

Game G_3*:* In this game hop, we catch the case, in which the forgery of adversary $\mathcal{A}$ contains a fresh and new triple $(m_1^*, \sigma_1^*, \mathsf{vk}^*)$ (see Line 4 of game G_3). We call this event E_2. Note that this case (together with **return** $\mathsf{Check}_{\mathsf{sg}}^{\Pi}()$ in Line 5) also catches the case in which UKE is broken. We bound the probability of this case to happen using reduction $\mathcal{R}_1$ to the sEUF-aCMA security of Σ. Besides that, the game hop from G_2 to G_3 does not differ from sg-$(\mathrm{aCMA}, \mathcal{O}_L)$ and it holds that

$$\Pr[\mathsf{G}_3(\lambda) = 1] = \Pr[\mathsf{G}_3(\lambda) = 1 \wedge E_2] + \Pr[\mathsf{G}_3(\lambda) = 1 \wedge \neg E_2].$$

Claim 2. *Let E_2 be the event in which G_3 aborts in Line 4. Then, $\Pr[\mathsf{G}_3(\lambda) = 1 \wedge E_2]$ occurs only with negligible probability $\epsilon_1(\lambda)$.*

Proof (of Claim 2). We build a reduction $\mathcal{R}_1$ that uses $\mathcal{A}$ from G_3 triggering an abort on E_2. I.e., $\mathcal{A}$ outputs a forgery that contains a fresh and new triple $(m_1^*, \sigma_1^*, \mathsf{vk}^*)$. We show how to leverage the capabilities of this adversary to break the sEUF-aCMA security of Σ. The reduction $\mathcal{R}_1$ receives as input a public key pk for Σ and has access to a signing oracle $\mathcal{O}_{\mathsf{Sign}}$ for Σ, which allows for adaptive chosen messages. Using this public key and the oracle, $\mathcal{R}_1$ simulates Game G_3 as follows (see Figure 14 for pseudocode). It forwards pk to $\mathcal{A}$ and provides access to the oracles $\mathcal{O}_1, \mathcal{O}_2$, and $\mathcal{O}_L$. To simulate $\mathcal{O}_1$, reduction $\mathcal{R}_1$ follows the description of G_3, but instead of signing message–temporary-key pairs (Line 3 of GenTsk, Figure 14), $\mathcal{R}_1$ uses the provided signing oracle. The same holds for $\mathcal{O}_2$, but using the verification algorithm of Σ. The reduction $\mathcal{R}_1$ simulates the leakage oracle $\mathcal{O}_L$ for $\mathcal{A}$ by storing each tsk in a queue $\mathcal{Q}_L$. On query (m, σ) to $\mathcal{O}_L$ from $\mathcal{A}$

reduction $\mathcal{R}_1$ checks if there exists a corresponding temporary key tsk in $\mathcal{O}_L$ and outputs this key. Otherwise, $\mathcal{R}_1$ outputs the empty set. This simulation of $\mathcal{R}_1$ is perfect, since the provided public key and signing oracle of Σ's sEUF-aCMA game are perfect simulations. Eventually, $\mathcal{A}$ outputs a forgery $(m_1^*, \sigma_1^*, \sigma_2^*)$. By the assumed abort condition, the pair $(m_1^* || \mathsf{vk}^*, \sigma_1^*)$ is a valid message–signature pair under pk. Because $(m_1^*, \sigma_1^*, \mathsf{vk}^*) \notin \mathcal{Q}_\Sigma$, this signature is new to $\mathcal{R}_1$. Thus, $\mathcal{R}_1$ outputs the forgery $(m_1^* || \mathsf{vk}^*, \sigma_1^*)$, which is a new forgery for Σ. This forgery contradicts our assumption of Σ being sEUF-aCMA secure. This implies, that $\epsilon_1(\lambda)$ is negligible. □

Finally, we can analyse the security of game sg-$(\text{aCMA}, \mathcal{O}_L)$ according to game G_3 under the assumption that the output message m_1^* and the challenge key vk are non-fresh (i.e., game G_3 did not abort). We show that the winning probability of an adversary in G_3 is bound by the winning probability of an adversary breaking sg under aCMA attacks.

Claim 3. *Let $\epsilon_2(\lambda)$ be the probability that an adversary $\mathcal{A}$ wins in game G_3 without causing Event E_2 or E_1. Let $\Pi = (\mathsf{A}_0, \mathsf{A}_1, \mathsf{A}_2)$ be a cryptographic scheme w.r.t. Model 3.1 satisfying a security goal* sg *against any* PPT *adversary $\mathcal{A}$ with adaptive chosen input attack* aCMA *against algorithm A_1 and corresponding public verification algorithm A_2. Then,* $\Pr[\mathsf{G}_3(\lambda) = 1 \wedge \neg E_1 \wedge \neg E_2] = \epsilon_2(\lambda)$ *is negligible.*

Proof (of Claim 3). We construct a reduction $\mathcal{R}_2$ that uses an adversary $\mathcal{A}$ winning in game G_3 to break the security goal sg of the scheme Π. Reduction $\mathcal{R}_2$ receives a public key pk^* and has access to an oracle $\mathcal{O}_{\mathsf{A}_1}$. It simulates G_3 for $\mathcal{A}$ as follows (see Figure 14 for pseudocode). First, $\mathcal{R}_2$ generates its own key pair $(\mathsf{sk}, \mathsf{pk})$ for the signature scheme Σ and provides pk to $\mathcal{A}$. When $\mathcal{A}$ makes oracle queries, $\mathcal{R}_2$ responds as follows: For a query $(m_i, \perp)$ from $\mathcal{A}$ to $\mathcal{O}_1$ (indicating a non-challenge request), reduction $\mathcal{R}_2$ simulates it honestly as described in G_3. For any query to $\mathcal{O}_L$, reduction $\mathcal{R}_2$ simulates it honestly as described in G_3 as long as the query does not correspond to the challenge instance, since it knows all other temporary secret keys. In the case of the challenge instance, $\mathcal{R}_2$ simply outputs the empty set. This is a perfect simulation of $\mathcal{O}_L$. For a query $(m_i, \top)$ from $\mathcal{A}$ to $\mathcal{O}_1$ (indicating a challenge request from $\mathcal{A}$), reduction $\mathcal{R}_2$ embeds its own challenge by setting the ephemeral verification key $\mathsf{vk}^* := \mathsf{pk}^*$ (i.e., the public key provided as input to the reduction) and querying its own oracle $\mathcal{O}_{\mathsf{A}_1}$ to obtain the output σ. Reduction $\mathcal{R}_2$ computes the signature σ' from Σ on $m_i || \mathsf{vk}^*$ by using its own key sk.

Eventually, $\mathcal{A}$ outputs a forgery $(m_1^*, \sigma_1^*, \sigma_2^*)$, which breaks sg of Π under a key-exposure attack. Since $\mathcal{R}_2$ embedded its own challenge by setting $\mathsf{vk}^* = \mathsf{pk}^*$, the adversary's winning output is precisely a valid solution for $\mathcal{R}_2$'s own sg-security game (without key exposure attack, as we simulated the key exposure oracle locally). $\mathcal{R}_2$ can then use this output to break sg from Π. Since this contradicts the assumption of our theorem, this implies that $\epsilon_2(\lambda)$ must be negligible. □

$\mathcal{R}_1^{\mathcal{O}_{\mathsf{Sign}}}(\mathsf{pk})$

$\mathsf{vk}^*,\ \mathcal{Q}_\Sigma,\ \mathcal{Q}_L \leftarrow \emptyset$
$(m_1^*, \sigma_1^*, \sigma_2^*) \leftarrow\$ \mathcal{A}^{\mathcal{O}_1,\mathcal{O}_2,\mathcal{O}_L}(\lambda, \mathsf{pk})$
return $(m_1^* || \mathsf{vk}^*, \sigma_1^*)$

$\mathcal{O}_1(m, b)$

$r \leftarrow\$ \mathcal{D}_R$
$(\mathsf{tsk}, \mathsf{vk}, \sigma') \leftarrow \mathsf{GenTsk}(m)$
$\sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$
if $b = \top \wedge \mathsf{vk}^* = \emptyset$:
  $\mathsf{vk}^* \leftarrow \mathsf{vk}$
else :
  $\mathcal{Q}_L \leftarrow \mathcal{Q}_L \cup \{m, \sigma, \mathsf{tsk}\}$
return $(\mathsf{vk}, \sigma, \sigma')$

$\mathcal{O}_2(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma')$

$b' \leftarrow \mathsf{Vrfy}(\mathsf{pk}, m || \mathsf{vk}, \sigma')$
$b \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$
return $b \wedge b'$

$\mathsf{GenTsk}(m)$

$r \leftarrow\$ \mathcal{D}_\lambda$
$\mathsf{tsk}, \mathsf{vk} \leftarrow \mathsf{A}_0(r)$
$\sigma \leftarrow \mathcal{O}_{\mathsf{Sign}}(m || \mathsf{vk})$
$\mathcal{Q}_\Sigma \leftarrow \mathcal{Q}_\Sigma \cup \{(m, \sigma, \mathsf{vk})\}$
return $(\mathsf{tsk}, \mathsf{vk}, \sigma)$

$\mathcal{O}_L(m, \sigma)$

$t = \{(m', \sigma', \mathsf{tsk}') \in \mathcal{Q}_L$
  $\mid m' = m \wedge \sigma' = \sigma\}$
return t

$\mathcal{R}_2^{\mathcal{O}_{\mathsf{A}_1}}(\mathsf{pk}^*)$

$(\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \mathsf{KGen}(1^\lambda)$
$\mathsf{vk}^*,\ \mathcal{Q}_\Sigma,\ \mathcal{Q}_L \leftarrow \emptyset$
$(m_1^*, \sigma_1^*, \sigma_2^*) \leftarrow\$ \mathcal{A}^{\mathcal{O}_1,\mathcal{O}_2,\mathcal{O}_L}(\lambda, \mathsf{pk})$
return (m_1^*, σ_2^*)

$\mathcal{O}_1(\mathsf{sk}, m, b)$

$r \leftarrow\$ \mathcal{D}_R$
if $b = \top \wedge \mathsf{vk}^* = \emptyset$:
  $\mathsf{vk}^* \leftarrow \mathsf{pk}^*;\ \mathsf{vk} \leftarrow \mathsf{pk}^*$
  $\sigma' \leftarrow \mathsf{Sign}(\mathsf{sk}, m || \mathsf{vk}^*)$
  $\sigma \leftarrow \mathcal{O}_{A_1}(m)$
else :
  $(\mathsf{tsk}, \mathsf{vk}, \sigma') \leftarrow \mathsf{GenTsk}(\mathsf{sk}, m)$
  $\sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$
  $\mathcal{Q}_L \leftarrow \mathcal{Q}_L \cup \{m, \sigma, \mathsf{tsk}\}$
return $(\mathsf{vk}, \sigma, \sigma')$

$\mathcal{O}_2(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma')$

$b' \leftarrow \mathsf{Vrfy}(\mathsf{pk}, m || \mathsf{vk}, \sigma')$
$b \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$
return $b \wedge b'$

$\mathsf{GenTsk}(\mathsf{sk}, m)$

$r \leftarrow\$ \mathcal{D}_\lambda$
$\mathsf{tsk}, \mathsf{vk} \leftarrow \mathsf{A}_0(r)$
$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m || \mathsf{vk})$
$\mathcal{Q}_\Sigma \leftarrow \mathcal{Q}_\Sigma \cup \{(m, \sigma, \mathsf{vk})\}$
return $(\mathsf{tsk}, \mathsf{vk}, \sigma)$

$\mathcal{O}_L(m, \sigma)$

$t = \{(m', \sigma', \mathsf{tsk}') \in \mathcal{Q}_L$
  $\mid m' = m \wedge \sigma' = \sigma\}$
return t

Fig. 14: Overview of reduction $\mathcal{R}_1$ and $\mathcal{R}_2$.

We can now conclude our proof. As G_3 simulates sg-(aCMA, $\mathcal{O}_L$) perfectly, if it does not abort, we have

$$\Pr[\mathsf{sg} - (\mathrm{aCMA}, \mathcal{O}_L) = 1] \leq \frac{q}{|\mathcal{D}_\lambda^{\mathsf{pk}}|} + \epsilon_1(\lambda) + \epsilon_2(\lambda),$$

which is negligible. □

```
GenTsk(sk, 𝕄)
1 :  if |𝕄| = 0 :
2 :     return ⊥
3 :  r ←$ 𝒟_R
4 :  tsk, vk ←$ A_0(r)
5 :  H ← BuildMerkleTree(𝕄)
6 :  R ← H[0]
7 :  σ' ← Sign(sk, R||vk)
8 :  return (tsk, vk, σ', R, H)

VTsk(pk, vk, σ', R, m, σ, π)
1 :  b_1 ← VrfyPath(R, m, π)
2 :  b_2 ← Vrfy(pk, R||vk, σ')
3 :  b_3 ← A_2(vk, m, σ)
4 :  if (b_1 = 1 ∧ b_2 = 1 ∧ b_3 = 1) :
5 :     return 1
6 :  return 0

VrfyPath(R, m, π)
1 :  h ← H(m)
2 :  for (b, s) ∈ π do :
3 :     if (b = 0) : h ← H(s||h)
4 :     else : h ← H(h||s)
5 :  return (h = R)

BuildMerkleTree(𝕄)
1 :  L ← [H(m) | m ∈ 𝕄]; t ← 1
2 :  if |L| = 0 : return [ ]
3 :  while (t < |L|) do : t ← 2 · t
4 :  while (|L| < t) do : L ← L||L[−1]
5 :  ℒ ← [L]
6 :  while |L| > 1 do :
7 :     L' ← [ ]
8 :     for i ∈ {0, 2, ..., |L| − 2} do :
9 :        L' ← L'||H(L[i]||L[i + 1])
10 :    ℒ ← ℒ||[L']; L ← L'
11 : H ← [ ]
12 : for i from |ℒ| − 1 to 0 do : H ← H||ℒ[i]
13 : return H

BuildAuthPath(H, m)
1 :  if |H| = 0 : return [ ]
2 :  h ← H(m); i ← −1
3 :  n ← (|H| + 1)/2; s ← |H| − n
4 :  for j ∈ {s, ..., |H| − 1} do :
5 :     if H[j] = h : i ← j; break
6 :  if i = −1 : return [ ]
7 :  π ← [ ]
8 :  while i > 0 do :
9 :     if (i mod 2 = 0) : π ← π||[(0, H[i − 1])]
10 :    else : π ← π||[(1, H[i + 1])]
11 :    i ← ⌊(i − 1)/2⌋
12 : return π
```

Fig. 15: Hash tree–based security compiler with authentication paths for key exposure. $\mathcal{L}$ stores per-level node lists during construction. L is the current level. Leaves are padded by duplicating the last to the next power of two. H is the level-order array with root R. $\mathsf{BuildAuthPath}(H, m)$ returns the path π; VTsk checks $\mathsf{VrfyPath}(R, m, \pi)$ and verifies σ' on $R||\mathsf{vk}$.

A Message Set Compiler. We now aim to enable offline-online phases for our compiler in order to enhance practicability and reduce the number of signatures GenTsk must produce. To this end, we employ Merkle trees [62]: rather than signing a single message hash, we sign the root R of a Merkle tree built over a set of messages, and for each message we provide its authentication path π from the leaf to R. This path-based approach is more optimal than sending or recomputing the full tree: it reduces verifier work and communication from $\mathcal{O}(|\mathbb{M}|)$ to $\mathcal{O}(\log|\mathbb{M}|)$ (in Landau notation), requires no state beyond R, and avoids distributing H entirely, while preserving the same security guarantees. Note that GenTsk could compute all paths for each message in $\mathbb{M}$. But to simplify

$\mathsf{Setup}(\mathsf{sk}, \mathbb{M})$	$\mathsf{A}_1'(\mathsf{tsk}, \mathsf{vk}, H, m)$	$\mathsf{A}_2'(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma', R, \pi)$
$1: \ \textbf{if } \lvert\mathbb{M}\rvert = 0 : \textbf{return } \perp$	$1: \ \pi \leftarrow \mathsf{BuildAuthPath}(H, m)$	$1: \ b_1 \leftarrow \mathsf{VrfyPath}(R, m, \pi)$
$2: \ r \leftarrow\$ \mathcal{D}_\lambda$	$2: \ \textbf{if } \lvert\pi\rvert = 0 : \textbf{return } \perp$	$2: \ b_2 \leftarrow \mathsf{Vrfy}(\mathsf{pk}, R\|\|\mathsf{vk}, \sigma')$
$3: \ (\mathsf{tsk}, \mathsf{vk}) \leftarrow \mathsf{A}_0(r)$	$3: \ r \leftarrow\$ \mathcal{D}_R$	$3: \ b_3 \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$
$4: \ H \leftarrow \mathsf{BuildMerkleTree}(\mathbb{M})$	$4: \ \sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$	$4: \ \textbf{return } b_1 \wedge b_2 \wedge b_3$
$5: \ R \leftarrow H[0]$	$5: \ \textbf{return } (\sigma, \pi)$	
$6: \ \sigma' \leftarrow \mathsf{Sign}(\mathsf{sk}, R\|\|\mathsf{vk})$		
$7: \ \textbf{return } (\mathsf{tsk}, \mathsf{vk}, \sigma', R, H)$		

Fig. 16: Overview of the tree-based key exposure security compiler.

the description, we compute the path with $\mathsf{BuildAuthPath}$ in A_1. Further, batching messages in this way is particularly meaningful in applications with a limited message space [8,59], and it remains compatible with aCMA security: While the Merkle tree may contain precomputed ephemeral signing keys for all messages in the space, an adversary might still adaptively select messages to obtain valid signatures; the verifier only checks membership via π and a single signature on R. Figure 15 (in **??**) shows the resulting construction for the Merkle tree and GenTsk. We provide the concrete adaptation of Figure 12 with Figure 15 in Figure 16. The main difference is that instead of single messages, this compiler digests message sets. We emphasize this with the explicit Setup algorithm, which allows us to outsource the computation of the hash tree and explicitly handle individual messages with A_1'.

Theorem 5 (Merkle Tree Compiler). *Let $\Pi = (\mathsf{A}_0, \mathsf{A}_1, \mathsf{A}_2)$ be a cryptographic scheme w.r.t. Model 3.1 satisfying a security goal* sg *against any* PPT *adversary $\mathcal{A}$ with adaptive chosen input attack* aCMA *against algorithm A_1 and corresponding public verification algorithm A_2. Let $\Sigma = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ be a* sEUF-aCMA *secure signature scheme. Let* GenTsk *be the security compiler w.r.t. our Model in Section 3.1. Let* H *be a collision resistant hash function. Let $\Pi' = (\mathsf{A}_0, \mathsf{A}_1', \mathsf{A}_2')$ be the scheme obtained from Π by applying the security compiler from Figure 16. Then, Π' satisfies* sg *with* UKE *against any* PPT *adversary $\mathcal{A}$ with* aCMA *and a key exposure oracle.*

Proof. We begin by providing a high-level overview of our game-hopping proof. We provide an overview of these four games in Figure 17. We provide an overview of the corresponding reductions $\mathcal{R}_0$, $\mathcal{R}_1$, and $\mathcal{R}_2$ in Figure 18. Again, our initial game G_1 is the original sg-(aCMA, $\mathcal{O}_L$) game, which includes our security compiler. Now, game G_2 catches the case in which there is a collision for the hash function. In the hop from G_1 to G_2, we argue that finding such a collision for a collision resistant hash function is negligible. Game G_3 catches an edge case, in which there is a collision between the main public key pk and the ephemeral verification keys vk. We bound such a collision (and hence, a gap between G_2 and G_3) to be negligible in the security parameter. Game G_4 catches the case in which the adversary outputs a forgery on a tuple containing a message–ephemeral-key pair. In the hop

$\mathsf{G}_1(\lambda), \mathsf{G}_2(\lambda), \mathsf{G}_3(\lambda), \mathsf{G}_4(\lambda)$

1 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \mathsf{Gen}(1^\lambda)$
2 : $\mathsf{vk}^*, \mathcal{Q}_\Sigma, \mathcal{Q}_L, \mathcal{Q}_R, \mathcal{Q}_H, \mathcal{Q}_{\mathsf{tsk}} \leftarrow \emptyset$
3 : $(m_1^*, \sigma_1^*, \sigma_2^*, R^*, \pi^*) \leftarrow\$ \mathcal{A}^{\mathcal{O}_{\mathsf{Setup}}, \mathcal{O}_1, \mathcal{O}_2, \mathcal{O}_L}(\lambda, \mathsf{pk})$
4 : **if** $\mathcal{Q}_R[R^*] \neq \emptyset \wedge \mathsf{VrfyPath}(R^*, m_1^*, \pi^*) = 1$ $\mathsf{G}_{\geq 2}$
5 : $\quad \wedge\ m_1^* \notin \mathcal{Q}_R[R^*]$: **abort** $\mathsf{G}_{\geq 2}$
6 : **if** $(R^*, \sigma_1^*, \mathsf{vk}^*) \notin \mathcal{Q}_\Sigma \wedge \mathsf{Vrfy}(\mathsf{pk}, R^* || \mathsf{vk}^*, \sigma_1^*)$: **abort** G_4
7 : **return** $\mathsf{Check}_{\mathsf{sg}}^{\Pi}()$

$\mathcal{O}_{\mathsf{Setup}}(\mathsf{sk}, \mathbb{M}, b)$

1 : **if** $|\mathbb{M}| = 0$: **return** $\perp$
2 : $r \leftarrow\$ \mathcal{D}_\lambda;\ (\mathsf{tsk}, \mathsf{vk}) \leftarrow \mathsf{A}_0(r)$
3 : **if** $\mathsf{pk} = \mathsf{vk}$: **abort** $\mathsf{G}_{\geq 3}$
4 : $H \leftarrow \mathsf{BuildMerkleTree}(\mathbb{M});$
5 : $R \leftarrow H[0];\ \mathcal{Q}_H[R] \leftarrow H$
6 : $\mathcal{Q}_R[R] \leftarrow \mathbb{M};\ \mathcal{Q}_{\mathsf{tsk}}[\mathsf{vk}] \leftarrow \mathsf{tsk}$
7 : $\sigma' \leftarrow \mathsf{Sign}(\mathsf{sk}, R || \mathsf{vk})$
8 : $\mathcal{Q}_\Sigma \leftarrow \mathcal{Q}_\Sigma \cup \{(R, \sigma', \mathsf{vk})\}$
9 : **if** $b = \top \wedge \mathsf{vk}^* = \emptyset : \mathsf{vk}^* \leftarrow \mathsf{vk}$
10 : **return** $(\perp, \mathsf{vk}, \sigma', R, H)$

$\mathcal{O}_1(\mathsf{tsk}, \mathsf{vk}, H, m)$

1 : $\pi \leftarrow \mathsf{BuildAuthPath}(H, m)$
2 : **if** $|\pi| = 0$:
3 : $\quad$ **return** $\perp$
4 : $r \leftarrow\$ \mathcal{D}_R;\ \sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$
5 : **if** $\mathsf{vk} \neq \mathsf{vk}^*$:
6 : $\quad \mathcal{Q}_L \leftarrow \mathcal{Q}_L \cup \{(m, \sigma, \mathsf{tsk})\}$
7 : **return** $(\mathsf{vk}, \sigma, \pi)$

$\mathcal{O}_2(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma', R, \pi)$

1 : $b_1 \leftarrow \mathsf{VrfyPath}(R, m, \pi)$
2 : $b_2 \leftarrow \mathsf{Vrfy}(\mathsf{pk}, R || \mathsf{vk}, \sigma')$
3 : $b_3 \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$
4 : **return** $b_1 \wedge b_2 \wedge b_3$

$\mathcal{O}_L(m, \sigma)$

1 : $t = \{(m', \sigma', \mathsf{tsk}') \in \mathcal{Q}_L$
2 : $\quad\quad | \ m' = m \wedge \sigma' = \sigma\}$
3 : **return** t

Fig. 17: Overview of the algorithms in the game hopping proof for the Merkle-tree compiler. $\mathsf{Check}_{\mathsf{sg}}^{\Pi}()$ is a generic function that has as input the full transcript of the security game and outputs 1 iff adversary $\mathcal{A}$ breaks the security goal sg. The oracle interface is $\mathcal{O}_{\mathsf{Setup}}(\mathsf{sk}, \cdot, \cdot)$, $\mathcal{O}_1(\mathsf{tsk}, \cdot, \cdot, \cdot)$, $\mathcal{O}_2(\mathsf{pk}, \cdot, \cdot, \cdot, \cdot, \cdot, \cdot)$, and $\mathcal{O}_L(\cdot, \cdot)$.

$\mathcal{R}_0^{\mathcal{O}_{\mathsf{H}}}(),\ \mathcal{R}_1^{\mathcal{O}_{\mathsf{Sign}}}(\mathsf{pk}),\ \mathcal{R}_2^{\mathcal{O}_{\mathsf{A}_1}}(\mathsf{pk}^*)$

1 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow\$ \mathsf{Gen}(1^\lambda)$ $\mathcal{R}_0, \mathcal{R}_2$
2 : $\mathsf{vk}^*, \mathcal{Q}_\Sigma, \mathcal{Q}_L, \mathcal{Q}_R, \mathcal{Q}_H, \mathcal{Q}_{\mathsf{tsk}} \leftarrow \emptyset$
3 : $(m_1^*, \sigma_1^*, \sigma_2^*, R^*, \pi^*) \leftarrow\$ \mathcal{A}^{\mathcal{O}_{\mathsf{Setup}}, \mathcal{O}_1, \mathcal{O}_2, \mathcal{O}_L}(\lambda, \mathsf{pk})$
4 : **return** (m', m_1^*) $\mathcal{R}_0$
5 : **return** $(R^* || \mathsf{vk}^*, \sigma_1^*)$ $\mathcal{R}_1$
6 : **return** (m_1^*, σ_2^*) $\mathcal{R}_2$

$\mathcal{O}_{\mathsf{Setup}}(\mathsf{sk}, \mathbb{M}, b)$

1 : **if** $|\mathbb{M}| = 0$: **return** $\perp$
2 : $r \leftarrow\$ \mathcal{D}_\lambda;\ (\mathsf{tsk}, \mathsf{vk}) \leftarrow \mathsf{A}_0(r)$
3 : **if** $b = \top \wedge \mathsf{vk}^* = \emptyset : \mathsf{vk} \leftarrow \mathsf{pk}^*$ $\mathcal{R}_2$
4 : $H \leftarrow \mathsf{BuildMerkleTree}(\mathbb{M});$
5 : $R \leftarrow H[0];\ \mathcal{Q}_H[R] \leftarrow H$
6 : $\mathcal{Q}_R[R] \leftarrow \mathbb{M};\ \mathcal{Q}_{\mathsf{tsk}}[\mathsf{vk}] \leftarrow \mathsf{tsk}$
7 : $\sigma' \leftarrow \mathcal{O}_{\mathsf{Sign}}(\mathsf{sk}, R || \mathsf{vk})$ $\mathcal{R}_1$
8 : $\sigma' \leftarrow \mathsf{Sign}(\mathsf{sk}, R || \mathsf{vk})$ $\mathcal{R}_0, \mathcal{R}_2$
9 : $\mathcal{Q}_\Sigma \leftarrow \mathcal{Q}_\Sigma \cup \{(R, \sigma', \mathsf{vk})\}$
10 : **if** $b = \top \wedge \mathsf{vk}^* = \emptyset : \mathsf{vk}^* \leftarrow \mathsf{vk}$
11 : **return** $(\perp, \mathsf{vk}, \sigma', R, H)$

$\mathcal{O}_1(\mathsf{tsk}, \mathsf{vk}, H, m)$

1 : $\mathsf{tsk} \leftarrow \mathcal{Q}_{\mathsf{tsk}}[\mathsf{vk}]$
2 : **if** $\mathsf{tsk} = \emptyset$: **return** $\perp$
3 : $\pi \leftarrow \mathsf{BuildAuthPath}(H, m)$
4 : **if** $|\pi| = 0$: **return** $\perp$
5 : $r \leftarrow\$ \mathcal{D}_R;$
6 : $\sigma \leftarrow \mathsf{A}_1(\mathsf{tsk}, m; r)$ $\mathcal{R}_0, \mathcal{R}_1$
7 : **if** $\mathsf{vk} = \mathsf{vk}^* : \sigma \leftarrow \mathcal{O}_{\mathsf{A}_1}(\mathsf{tsk}, m; r)$ $\mathcal{R}_2$
8 : **if** $\mathsf{vk} \neq \mathsf{vk}^*$:
9 : $\quad \mathcal{Q}_L \leftarrow \mathcal{Q}_L \cup \{(m, \sigma, \mathsf{tsk})\}$
10 : **return** $(\mathsf{vk}, \sigma, \pi)$

$\mathcal{O}_2(\mathsf{pk}, \mathsf{vk}, m, \sigma, \sigma', R, \pi)$

1 : $b_1 \leftarrow \mathsf{VrfyPath}(R, m, \pi)$
2 : $b_2 \leftarrow \mathsf{Vrfy}(\mathsf{pk}, R || \mathsf{vk}, \sigma')$
3 : $b_3 \leftarrow \mathsf{A}_2(\mathsf{vk}, m, \sigma)$
4 : **return** $b_1 \wedge b_2 \wedge b_3$

$\mathcal{O}_L(m, \sigma)$

1 : $t = \{(m', \sigma', \mathsf{tsk}') \in \mathcal{Q}_L$
2 : $\quad\quad | \ m' = m \wedge \sigma' = \sigma\}$
3 : **return** t

Fig. 18: Overview of the reductions $\mathcal{R}_0$, $\mathcal{R}_1$, and $\mathcal{R}_2$ for the proof of Theorem 5. We provide the computation of m' in Line 4 of $\mathcal{R}_0$ in the proof of Claim 4.

from G_3 to G_4, we argue that this forgery is only possible if the adversary breaks the sEUF-aCMA security of Σ, which contradicts our assumption. Consequently, we can finish the proof by analysing the sg-(aCMA, $\mathcal{O}_L$) of G_4 with a reduction to the security of Π satisfying security goal sg under an adaptively chosen input aCMA and key exposure attack to the security of Π satisfying sg under aCMA attacks without key exposure.

Game G_1*:* Our first game G_1 is the original sg-(aCMA, $\mathcal{O}_L$) game. It holds that $\Pr[\mathsf{sg}\text{-}(\text{aCMA}, \mathcal{O}_L)^{\mathcal{A}}_{\Pi}(\lambda) = 1] = \Pr[\mathsf{G}_1(\lambda) = 1]$. To simplify the notation of G_1, we introduce a generic function $\mathsf{Check}^{\Pi}_{\mathsf{sg}}()$ which has as input the full transcript of the security game and outputs 1 iff the actions of the adversary break the security goal sg.

Game G_2*:* In this game, we catch the case in which the adversary's forgery contains a message m_1^* such that the Merkle-path verification on any of our roots R succeeds, i.e., $\mathsf{VrfyPath}(R, m_1^*, \pi^*) = 1$, but $m_1^* \notin \mathcal{Q}_R[R]$ (see Line 6 and 7 of Figure 17). We call this event E_3. Besides that, the game hop from G_1 to G_2 does not differ from sg-(aCMA, $\mathcal{O}_L$) and it holds that

$$\Pr[\mathsf{G}_2(\lambda) = 1] = \Pr[\mathsf{G}_2(\lambda) = 1 \wedge E_3] + \Pr[\mathsf{G}_2(\lambda) = 1 \wedge \neg E_3].$$

Claim 4. *Let E_3 be the event in which G_2 aborts in Lines 6–7 of Figure 17. Let H be a collision resistant hash function. Then, $\Pr[\mathsf{G}_2(\lambda) = 1 \wedge E_3]$ occurs only with negligible probability $\epsilon_3(\lambda)$.*

Proof (of Claim 4). We construct a reduction $\mathcal{R}_0$ with an oracle $\mathcal{O}_{\mathsf{H}}$ for the hash function that uses an adversary $\mathcal{A}$ causing the abort in G_2 (event E_3) to find a collision for H. $\mathcal{R}_0$ simulates the sg-(aCMA, $\mathcal{O}_L$) experiment according to the description in Figure 18. This is equivalent to the game description in Figure 17, except for one major difference. The adversary should not receive the temporary secret keys besides utilizing the leakage oracle, i.e., $\mathcal{O}_{\mathsf{Setup}}$ can not simply output tsk. However, the reduction still needs to get the correct tsk for a corresponding verification key vk in $\mathcal{O}_1$. To do so, the reduction makes bookkeeping utilizing $\mathcal{Q}_{\mathsf{tsk}}$ and accesses the necessary tsk in Line 1 of $\mathcal{O}_1$. This bookkeeping does not affect the simulation. Eventually, $\mathcal{A}$ outputs $(m_1^*, \sigma_1^*, \sigma_2^*, \pi^*)$ with $\sigma_2^* = (\mathsf{vk}^*, \sigma'^*, R^*)$: According to the assumed abort condition, we have that $\mathcal{Q}_R[R^*] \neq \emptyset$ (i.e., R^* is a signed root), $\mathsf{VrfyPath}(R^*, m_1^*, \pi^*) = 1$ (i.e., the path verification accepts m_1^*), and $m_1^* \notin \mathcal{Q}_R[R^*]$ (i.e., m_1^* is not part of the corresponding Merkle tree). Let H^* be the Merkle tree for $\mathcal{Q}_R[R^*]$. Since $\mathsf{VrfyPath}(R^*, m_1^*, \pi^*) = 1$ holds while $m_1^* \notin \mathcal{Q}_R[R^*]$, there is level j on the path verification where the "original" pair (x, y) in H^* differs from the "new" pair (x', y') implied by (m_1^*, π^*). At that level we have $\mathsf{H}(x||y) = \mathsf{H}(x'||y')$ with $(x, y) \neq (x', y')$. This is a collision for the hash function H, and reduction $\mathcal{R}_0$ can output $(x||y, x'||y')$. Due to the collision resistance of H in the random-oracle model, the probability $\epsilon_3(\lambda)$ is negligible. □

Game G_3*:* This game only differs from G_2 if the ephemeral verification key vk equals the main public key pk. We call this event E_1.

Claim 5. *Let E_1 be the event G_3 aborts. Then,* $\Pr[\mathsf{G}_3(\lambda) = 1 \wedge E_1]$ *is negligible.*

The proof of Claim 5 is analogous to the proof of Claim 1.

Game G_4*:* This game catches the case, in which the forgery of adversary $\mathcal{A}$ contains a fresh and new triple $(m_1^*, \sigma_1^*, \mathsf{vk}^*)$. We call this event E_2.

Claim 6. *Let E_2 be the event G_4 aborts. Then,* $\Pr[\mathsf{G}_4(\lambda) = 1 \wedge E_2]$ *is negligible.*

The proof of Claim 6 is analogous to the proof of Claim 2. Finally, we can analyse the security of game sg-$(\mathrm{aCMA}, \mathcal{O}_L)$ according to game G_4 under the assumptions that the output message m_1^* and the challenge key vk are non-fresh as well as having no collision for the hash function building the Merkle tree. We can show that the winning probability of an adversary in G_4 is bound by the winning probability of an adversary breaking sg under aCMA attacks.

Claim 7. *Let $\Pi = (\mathsf{A}_0, \mathsf{A}_1, \mathsf{A}_2)$ be a cryptographic scheme w.r.t. Model 3.1 satisfying a security goal* sg *against any* PPT *adversary $\mathcal{A}$ with adaptive chosen input attack* aCMA *against algorithm* A_1 *and corresponding public verification algorithm* A_2*. Then,* $\Pr[\mathsf{G}_3(\lambda) = 1 \wedge \neg E_1 \wedge \neg E_2 \wedge \neg E_3]$ *is negligible.*

The proof of Claim 7 is analogous to the proof of Claim 3. This concludes our proof and gives an overall negligible bound. □

E Adapting Schnorr Signatures

This section brings together our security compilers of Section C as well as Section D within the security framework of Section 4.2. In particular, we apply both compilers to the Schnorr signature scheme [71] to achieve security against adaptive chosen randomness attacks (aCRA) and key exposure, while still being secure against adaptively chosen message attacks (aCMA). We chose the fundamental and widely used Schnorr scheme to show how to merge the generic solutions for aCMA with key exposure and aCMA with aCRA on a practical example. First, we provide the corresponding security definition as a specification of the framework from Section 4.2.

Definition 10 ($(\mathrm{sEUF}, \mathrm{UKE})$-$(\mathrm{aCMA}, \mathrm{aCRA}, L_{\mathsf{tsk}}^{m,\sigma})$-Security). *Let Σ be a Signature scheme. Σ achieves* $(\mathrm{sEUF}, \mathrm{UKE})$-$(\mathrm{aCMA}, \mathrm{aCRA}, L_{\mathsf{tsk}}^{m,\sigma})$*-security, if for all probabilistic polynomial time* PPT *adversaries $\mathcal{A}$*

$$\Pr[(\mathrm{sEUF}, \mathrm{UKE}) - (\mathrm{aCMA}, \mathrm{aCRA}, L_{\mathsf{tsk}}^{m,\sigma})_{\Sigma}^{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$$

holds, where negl *is a negligible function. The probability is taken over the randomness used by $\mathcal{A}$ and the randomness used in the experiment.*

The idea is to adapt the Schnorr signature scheme by integrating the security compilers GenTr from Section C and GenTsk from Section D. In the following, we rely on the results of these sections. For the PRF of GenTr, we use two hash

functions H_1 and H_2 in the random oracle model, since the Schnorr signature scheme already utilizes a random oracle. We use H_1 for the randomness within GenTsk and H_2 for the Schnorr scheme. We separate both hash functions by using domain separation.

GenTsk derives the temporary secret keys to protect against key leakage. We instantiate the hash function for the Merkle tree construction with a third (domain-separated) random oracle H_3 in the same manner as for GenTr. Additionally, we introduce an algorithm Setup_{AS} for the Merkle tree optimization. We assume that the temporary secret keys from Setup_{AS} are securely stored and are not directly sent to an adversary. Similarly, we assume that Sign_{AS} can securely access the needed temporary secret keys tsk for the corresponding input. We formally present the original Schnorr signature scheme Σ_S in Construction 1. Then we present the adapted signature scheme Σ_{AS} in Construction 2 and state the corresponding improved security in Theorem 6, which we prove thereafter.

Construction 1 (Schnorr Signature Scheme) *Let $\mathbb{G}$ be a group of order p with generator g. Let $\mathsf{H}\colon \mathbb{G} \times \{0,1\}^* \to \mathbb{Z}_p$ be a hash function. The Schnorr signature scheme $\Sigma_S = (\mathsf{KGen}_S, \mathsf{Sign}_S, \mathsf{Vrfy}_S)$ consists of the following algorithms:*

$\mathsf{KGen}_S(r)$*: The key generation algorithm takes as input randomness r, derives $\mathsf{sk} \leftarrow\$ \mathbb{Z}_p$ based on r, computes the public key $\mathsf{pk} \leftarrow g^{\mathsf{sk}}$, and outputs $(\mathsf{sk}, \mathsf{pk})$.*

$\mathsf{Sign}_S(\mathsf{sk}, m; r)$*: The signing algorithm takes as input a secret key sk, a message m, and a random nonce $r \in \mathbb{Z}_p$. It computes commitment $C_r \leftarrow g^r$, challenge $c \leftarrow \mathsf{H}(\mathsf{pk}, C_r, m)$, response $y \leftarrow r + \mathsf{sk} \cdot c \mod p$, and outputs $\sigma_S = (y, C_r)$.*

$\mathsf{Vrfy}_S(\mathsf{pk}, m, \sigma_S)$*: The verification algorithm takes as input a public key pk, a message m, and a signature $\sigma_S = (y, C_r)$. It computes $c \leftarrow \mathsf{H}(\mathsf{pk}, C_r, m)$ and outputs 1 (accepts), if $g^y = C_r \cdot \mathsf{pk}^c$ holds. Otherwise, it outputs 0 (rejects).*

Construction 2 (Adapted Schnorr Signature Scheme) *Let $\mathbb{G}$ be a group of prime order p with generator g. Let H_1, H_2, and H_3 be hash functions. Let $\Sigma_{\mathrm{aCMA}} = (\mathsf{KGen}_{\mathrm{aCMA}}, \mathsf{Sign}_{\mathrm{aCMA}}, \mathsf{Vrfy}_{\mathrm{aCMA}})$ be a signature scheme and $\Sigma_S = (\mathsf{KGen}_S, \mathsf{Sign}_S, \mathsf{Vrfy}_S)$ be the Schnorr signature scheme. The adapted Schnorr signature scheme $\Sigma_{AS} = (\mathsf{KGen}_{AS}, \mathsf{Setup}_{AS}, \mathsf{Sign}_{AS}, \mathsf{Vrfy}_{AS})$ consists of the following efficient algorithms:*

$\mathsf{KGen}_{AS}(r)$*: The key generation algorithm takes as input randomness r, computes $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}_S(r)$ and outputs $(\mathsf{sk}, \mathsf{pk})$.*

$\mathsf{Setup}_{AS}(\mathsf{sk}, \mathbb{M}; r)$*: The setup algorithm takes as input a private key sk, a message set $\mathbb{M}$, and a random value r. It generates a Merkle tree H and outputs the tuple $(\mathsf{tsk}, \mathsf{vk}, \sigma_a, R, H)$ with temporary secret key tsk, corresponding verification key vk, signature σ_a, hash tree H with root $R := H[0]$. We define the computation in Figure 19.*

$\mathsf{Sign}_{AS}(\mathsf{tsk}, \mathsf{vk}, H, m; r)$*: The signing algorithm takes as input a temporary secret key tsk, verification key vk, a Merkle tree H, a message m, and a random element r. It outputs a signature σ_S and a proof π. We define the computation in Figure 19.*

$\mathsf{Setup}_{AS}(\mathsf{sk}, \mathbb{M}; r)$	$\mathsf{Sign}_{AS}(\mathsf{tsk}, \mathsf{vk}, H, m; r)$	$\mathsf{Vrfy}_{AS}(\mathsf{pk}, \mathsf{vk}, m, \sigma_S, \sigma_a, R, \pi)$
$1: \ \mathsf{tr}_1 \leftarrow \mathsf{H}_1(\mathsf{sk}, \mathbb{M}; r)$	$1: \ \pi \leftarrow \mathsf{BuildAuthPath}(H, m)$	$1: \ b_1 \leftarrow \mathsf{VrfyPath}(R, m, \pi)$
$2: \ \mathsf{tsk}, \mathsf{vk} \leftarrow \mathsf{KGen}_S(\mathsf{tr}_1)$	$2: \ \textbf{if } \lvert\pi\rvert = 0 : \textbf{return } \perp$	$2: \ b_2 \leftarrow \mathsf{Vrfy}_{\mathrm{aCMA}}(\mathsf{pk}, R \lVert \mathsf{vk}, \sigma_a)$
$3: \ H \leftarrow \mathsf{BuildMerkleTree}_{\mathsf{H}_3}(\mathbb{M})$	$3: \ \mathsf{tr}_2 \leftarrow \mathsf{H}_2(\mathsf{tsk}, m; r)$	$3: \ b_3 \leftarrow \mathsf{Vrfy}_S(\mathsf{vk}, m, \sigma_S)$
$4: \ R \leftarrow H[0]$	$4: \ \sigma_S \leftarrow \mathsf{Sign}_S(\mathsf{tsk}, m; \mathsf{tr}_2)$	$4: \ \textbf{return } b_1 \wedge b_2 \wedge b_3$
$5: \ \sigma_a \leftarrow \mathsf{Sign}_{\mathrm{aCMA}}(\mathsf{sk}, R \lVert \mathsf{vk})$	$5: \ \textbf{return } (\sigma_S, \pi)$	
$6: \ \textbf{return } (\mathsf{tsk}, \mathsf{vk}, \sigma_a, R, H)$		

Fig. 19: Description of Setup_{AS}, Sign_{AS}, and Vrfy_{AS} for Construction 2.

$\mathsf{Vrfy}_{AS}(\mathsf{pk}, \mathsf{vk}, m, \sigma_S, \sigma_a, R, \pi)$*: The verification algorithm takes as input a public key* pk*, a verification key* vk*, a message* m*, two signatures* σ_S *as well as* σ_{aCMA}*, a root* R*, and a proof* π*. It outputs a bit* b*, with* $b = 1$ *indicating accept and* $b = 0$ *indicating reject. We define the computation in Figure 19.*

Theorem 6 (Security of Construction 2). *Let* Σ_{aCMA} *be a strong existential unforgeable against adaptive chosen message (*sEUF-aCMA*) secure signature scheme. Let* Σ_S *be the strong existential unforgeable against adaptive chosen message (*sEUF-aCMA*) secure Schnorr signature scheme. If* H_1*,* H_2*, and* H_3 *are modeled as domain-separated random oracles, then the Signature scheme* Σ_{AS} *of Construction 2 achieves* $(\mathrm{sEUF}, \mathrm{UKE})$-$(\mathrm{aCMA}, \mathrm{aCRA}, L^{m,\sigma}_{\mathsf{tsk}})$*-security.*

Proof. We achieve the statement in the theorem with the following steps. We begin with the Schnorr signature scheme from Construction 1. We instantiate the Merkle tree compiler using H_3 in the random oracle model. By applying the Merkle tree compiler from Section D to Construction 1, we get an intermediate scheme $\Sigma'_{AS} = (\mathsf{KGen}'_{AS}, \mathsf{Setup}'_{AS}, \mathsf{Sign}'_{AS}, \mathsf{Vrfy}'_{AS})$. The random oracle model for H_3 implies collision resistance and Construction 1 is (sEUF)-(aCMA)-secure. Hence, by Theorem 5, the resulting construction Σ'_{AS} achieves $(\mathrm{sEUF}, \mathrm{UKE})$-$(\mathrm{aCMA}, L^{m,\sigma}_{\mathsf{tsk}})$-security. As a next step, we also apply our randomness compiler from Section C. Since a keyed random oracle behaves as a PRF, we can use $\mathsf{PRF}(\mathsf{sk}, \mathit{input}; r)$ from Theorem 3 to secure against aCRA attacks. This is possible since the Schnorr signature scheme requires different random values for different signatures, but not necessarily for the same message. This corresponds to our notion of weak non-repeatability. We apply this separately for Setup'_{AS} with H_1 and Sign'_{AS} with H_2 with domain separated hash functions/random oracles. Note that even if we have a temporary secret key tsk instead of a hidden secret key sk, the argument of Theorem 3 still applies. In the aCRA experiment, the adversary fixes r before learning tsk. Hence, the random-oracle output for (tsk, m, r) is hidden at signing time, and the resulting tr is uniformly distributed in $\mathbb{Z}_p$. Any leakage of tsk is only possible after signing m and therefore cannot retroactively bias or correlate tr. For any other message m' (on the same tsk), the output of the random oracle model is again uniformly distributed. The same holds for any other temporary secret key tsk'. Putting things together, by applying Theorem 3 to Σ'_{AS} to secure against aCRA, we achieve $(\mathrm{sEUF}, \mathrm{UKE})$-$(\mathrm{aCMA}, \mathrm{aCRA}, L^{m,\sigma}_{\mathsf{tsk}})$-security for the resulting scheme Σ_{AS}. □